



**SecureVault**

Secure Password Management System

# SECUREVAULT

Secure Password Management System

A zero-knowledge, web-based password and secure-document manager

PRG800 – Systems Analysis and Design

## Milestone 4 — System Design

Submitted to: Professor Miles McDonald

Submitted by: Secure Vault Group

PRG800NAA.06353.2264 • Seneca Polytechnic

| Name                              | Student # |
|-----------------------------------|-----------|
| Felix Akuetteh Torto              | 168365229 |
| Akshay Nedumparambil Unnikrishnan | 190635235 |
| Harmon Anthony Tuazon             | 165229220 |
| Jean Luc Sita Mbuya               | 111827242 |
| Kulsum Timol                      | 112867247 |
| Anju Babu                         | 115640245 |

June 30, 2026

## Table of Contents

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Executive Summary.....                                                     | 5  |
| Part I — Software Design & User Interface / User Experience .....          | 6  |
| 1. Software Design .....                                                   | 6  |
| 1.1 Design Approach and Architecture Patterns .....                        | 6  |
| 1.2 Design Class Diagram .....                                             | 6  |
| 1.3 Use Case Realizations — Sequence Diagrams .....                        | 8  |
| 2. User Interface / User Experience Design .....                           | 18 |
| 2.1 Design Principles Applied .....                                        | 18 |
| 2.2 Navigation Map.....                                                    | 19 |
| 2.3 Login and Two-Factor User Flow.....                                    | 19 |
| 2.4 Screen Wireframes.....                                                 | 21 |
| 3. Design-to-Requirements Traceability .....                               | 25 |
| 4. Refinements to Business Requirements .....                              | 25 |
| 5. Summary .....                                                           | 25 |
| Part II — Database Architecture .....                                      | 26 |
| 1. Database Choice & Naming Conventions .....                              | 26 |
| 2. Data Dictionary .....                                                   | 26 |
| 3. Entity Relationships & Cardinality.....                                 | 28 |
| 4. Data Definition Language (DDL).....                                     | 29 |
| 5. Design Notes .....                                                      | 32 |
| Part III — System Architecture & Security: Cloud Deployment & DevOps ..... | 33 |
| 1. Introduction & Executive Summary .....                                  | 33 |
| 2. Technology Stack .....                                                  | 33 |
| 3. Deployment Architecture .....                                           | 34 |
| 3.1 Architecture Overview .....                                            | 34 |
| 3.2 Component Responsibilities .....                                       | 35 |
| 3.3 Runtime Request Flow .....                                             | 35 |
| 3.4 Network & Security Boundaries.....                                     | 36 |
| 4. Provisioned Resources (Terraform) .....                                 | 36 |
| 4.1 Resource Inventory .....                                               | 36 |
| 4.2 Terraform Project Structure.....                                       | 37 |
| 4.3 Remote State & Locking.....                                            | 38 |

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 4.4 Cost-Aware Sizing.....                                      | 38 |
| 5. CI/CD Pipeline (GitHub Actions).....                         | 38 |
| 5.1 Pipeline Overview .....                                     | 38 |
| 5.2 Pipeline Stages .....                                       | 39 |
| 5.3 Deployment Strategy .....                                   | 39 |
| 5.4 Keyless Authentication (Workload Identity Federation) ..... | 40 |
| 6. Cost Analysis & Budget Management.....                       | 40 |
| 6.1 Estimated Monthly Cost.....                                 | 40 |
| 6.2 Cost Controls .....                                         | 41 |
| 7. Motivation & Rationale.....                                  | 41 |
| 8. Points of Consideration.....                                 | 42 |
| 8.1 Secrets Hosting .....                                       | 42 |
| 8.2 Security & Least Privilege.....                             | 42 |
| 8.3 Cost & Teardown.....                                        | 42 |
| 8.4 Backups & Recovery.....                                     | 42 |
| 8.5 Region & Data Residency .....                               | 42 |
| 8.6 Availability & Known Limits.....                            | 42 |
| 8.7 Risks & Mitigations.....                                    | 43 |
| 9. Summary .....                                                | 43 |
| References .....                                                | 43 |
| Part IV — System Test Plan .....                                | 45 |
| 1. Test Plan Identifier.....                                    | 46 |
| 2. References.....                                              | 46 |
| 3. Introduction .....                                           | 46 |
| 4. Test Items .....                                             | 47 |
| 5. Software Risk Issues.....                                    | 47 |
| 6. Features to be Tested .....                                  | 47 |
| 7. Features Not to be Tested.....                               | 48 |
| 8. Approach.....                                                | 49 |
| 8.1 Test Levels .....                                           | 49 |
| 8.2 Test Techniques .....                                       | 49 |
| 8.3 Test Tools .....                                            | 49 |
| 8.4 Metrics Collected .....                                     | 50 |

|                                                          |    |
|----------------------------------------------------------|----|
| 8.5 Configuration Management.....                        | 50 |
| 8.6 Regression Testing .....                             | 50 |
| 8.7 Special Requirements.....                            | 50 |
| 8.8 Organisational Processes .....                       | 51 |
| 9. Item Pass/Fail Criteria.....                          | 51 |
| 10. Suspension Criteria and Resumption Requirements..... | 51 |
| 11. Test Deliverables .....                              | 52 |
| 12. Remaining Test Tasks .....                           | 52 |
| 13. Test Cases.....                                      | 52 |
| 14. Environmental Needs.....                             | 60 |
| 15. Staffing and Training Needs .....                    | 60 |
| 16. Responsibilities .....                               | 61 |
| 17. Schedule .....                                       | 62 |
| 18. Planning Risks and Contingencies .....               | 63 |
| 19. Approvals .....                                      | 63 |
| 20. Glossary .....                                       | 64 |
| Summary .....                                            | 66 |

## Executive Summary

SecureVault is a zero-knowledge, web-based password management system that lets individual users store, generate, and manage credentials and sensitive documents inside an AES-256 encrypted vault, protected by a hashed master password and two-factor authentication. Milestones 1 through 3 established the project plan, feasibility, and the full requirements analysis — five core use cases, a domain model, and the non-functional requirements. This Milestone 4 document turns those requirements into a complete system design: it specifies how SecureVault will be built and how it will run.

The report is organised into four parts that together address every design consideration required for this milestone. Part I (Software Design & UI/UX) promotes the domain model into a layered architecture of controllers, a data-access layer, and a cryptographic service, realises all five use cases as sequence diagrams, and presents the user-interface design as a navigation map, a login-and-2FA flow, and six annotated wireframes. Part II (Database Architecture) maps the domain model to a normalised MySQL schema with a full data dictionary, an entity-relationship diagram, and the security and inheritance decisions behind it. Part III (System Architecture & Security) defines the cloud deployment architecture on Google Cloud Platform — the runtime topology, the infrastructure-as-code that provisions it, the CI/CD pipeline, and the network, identity, and secrets boundaries that enforce SecureVault's zero-knowledge posture. Part IV (System Test Plan) is an IEEE-829 test plan that verifies every use case and security requirement through 48 documented test cases.

Read together, the four parts cover the five areas the design must address: the system architecture and its security (the layered application architecture in Part I and the deployment architecture and security boundaries in Part III), the user interface and experience (Part I), the database architecture (Part II), the software design (Part I), and the system test plans (Part IV). Every design element traces back to a Milestone 3 use case or business rule, so the design is consistent, secure, and ready to guide implementation on the project's Node.js / Express and MySQL stack.

## Part I — Software Design & User Interface / User Experience

### 1. Software Design

Analysis produced a domain model of SecureVault’s concepts; design specifies the software classes, their operations, and the architecture that hosts them. This section covers the architectural approach and patterns, the design class diagram, and the five use-case realizations.

#### 1.1 Design Approach and Architecture Patterns

SecureVault uses a three-layer architecture: the presentation layer receives requests, the business-logic layer holds the domain classes and rules, and the data-access layer handles persistence. This keeps the security-critical logic isolated from both the interface and the database. A few well-known patterns give the design its structure:

| Pattern                        | Role in SecureVault                                                                                                             |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Three-layer architecture       | Separates Presentation, Business Logic, and Data Access so each can change independently and the security logic stays isolated. |
| MVC-style controllers          | AuthController, VaultController, and HealthController receive interface requests and orchestrate the domain classes.            |
| Data Access Object (DAO)       | UserDAO, VaultItemDAO, and AuditDAO isolate all SQL, keeping the domain classes free of persistence detail.                     |
| Cryptographic service (facade) | CryptoService is the single point for AES-256 encryption and password hashing, enforcing the zero-knowledge boundary.           |
| Stateless session token        | Session carries a signed, time-limited token that supports the ten-minute auto-lock and per-request validation.                 |
| Narrow external interfaces     | AuthenticationService and EmailService are reached through small, replaceable interfaces.                                       |

#### 1.2 Design Class Diagram

The design class diagram evolves the Milestone 3 model three ways: domain classes gain method signatures, types, and visibility markers (+ public, – private); design-only classes are added for the presentation and data-access layers; and a CryptoService names the encryption boundary the requirements imply. Classes are grouped by layer. The diagram keeps the UML conventions used in the course — multiplicity, hollow-triangle inheritance, and abstract classes shown in italics — and extends the analysis-level domain model with that design detail.

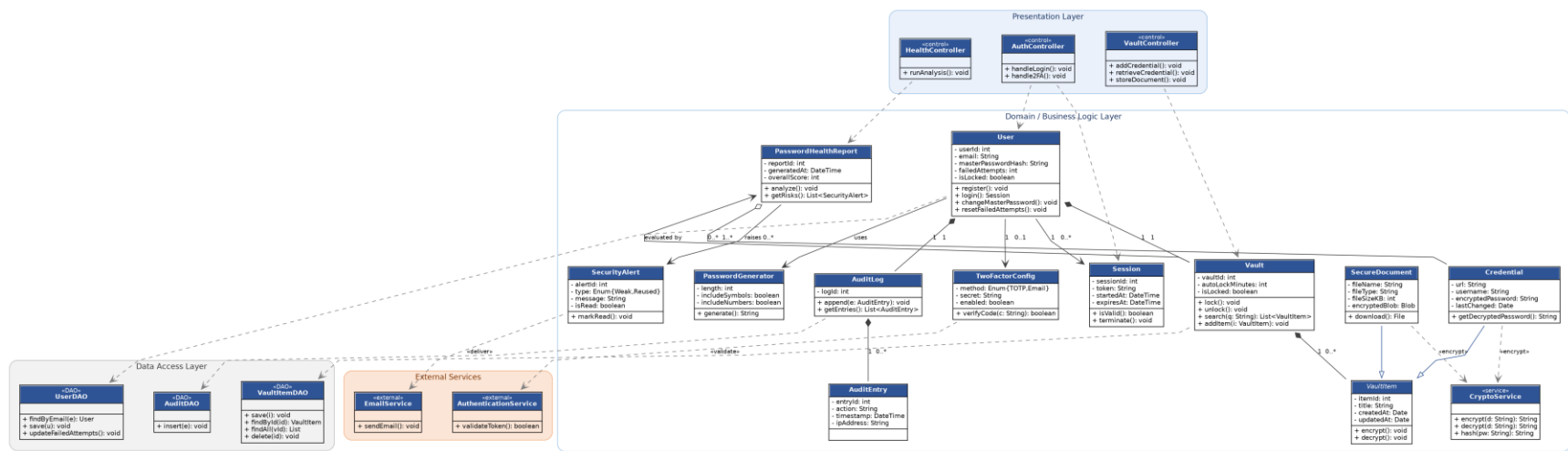


Figure 1. SecureVault design class diagram, grouped by architectural layer.

The analysis relationships are preserved: Credential and SecureDocument inherit from the abstract VaultItem; a User is composed of one Vault and one AuditLog; and a PasswordHealthReport aggregates the Credentials it evaluates. Dashed arrows show the design additions — controllers drive the domain classes, which persist through their DAOs and encrypt via the CryptoService, while the external auth and email services sit at the boundary.

### **1.3 Use Case Realizations — Sequence Diagrams**

Each use case is realized as a sequence diagram using the design classes from Figure 1. Solid arrows are calls, dashed arrows returns; «include» and «extend» match the Milestone 3 use case diagram. Exception paths are noted with each figure.

#### ***1.3.1 UC-01 Log In***

The AuthController checks the master password against its hash via the CryptoService, then the «extend» Verify 2FA step validates the second factor through the external AuthenticationService before a Session is created. Exceptions: an invalid password or code returns a generic failure and increments the attempt counter; five failures lock the account for fifteen minutes.



Sequence Diagram — UC-01 Log In (with Verify 2FA)

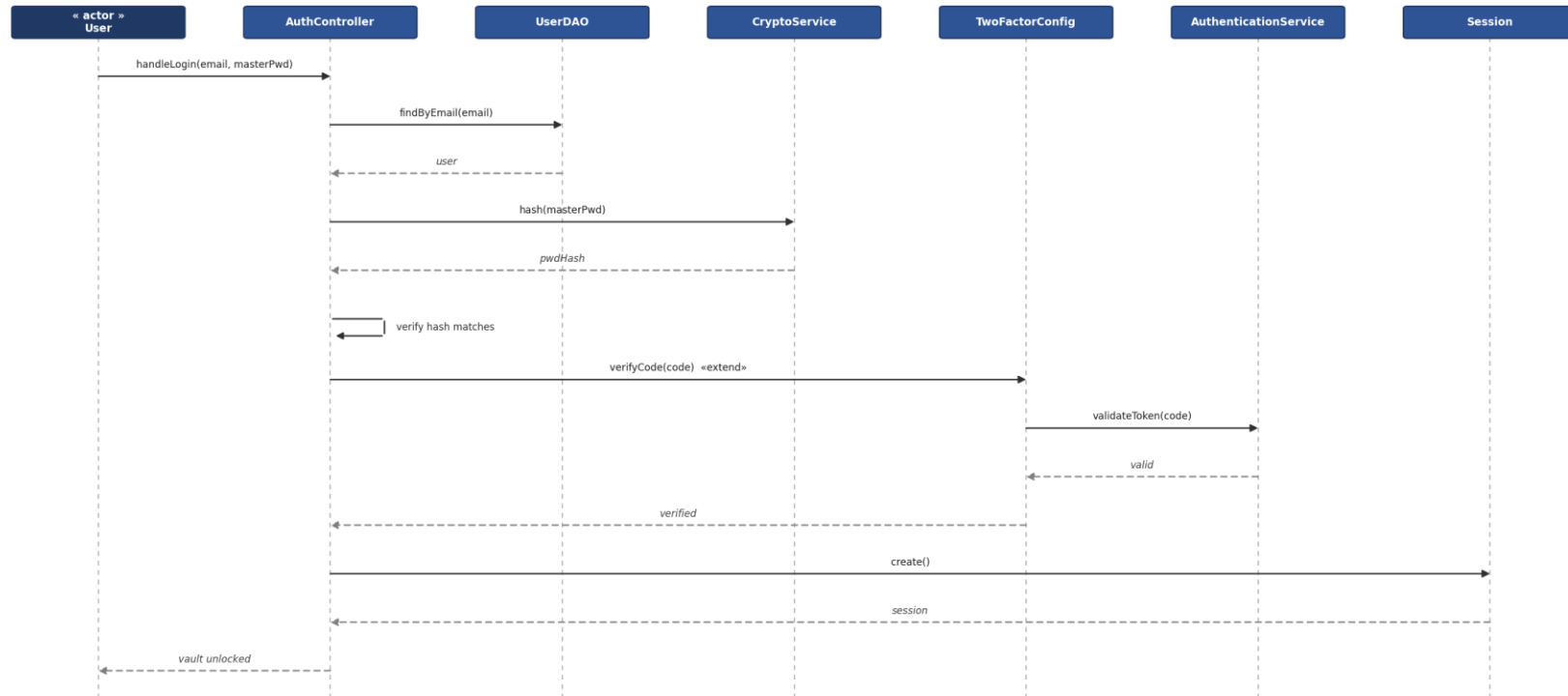


Figure 2. Sequence realization of UC-01 Log In, including the Verify 2FA extension.

### ***1.3.2 UC-02 Add Credential***

The VaultController validates the input, the «include» Encrypt Data step routes the password through the CryptoService, and the encrypted record is saved via the VaultItemDAO and logged. Exceptions: a missing field blocks the save; an encryption failure aborts without storing partial data.

Sequence Diagram — UC-02 Add Credential

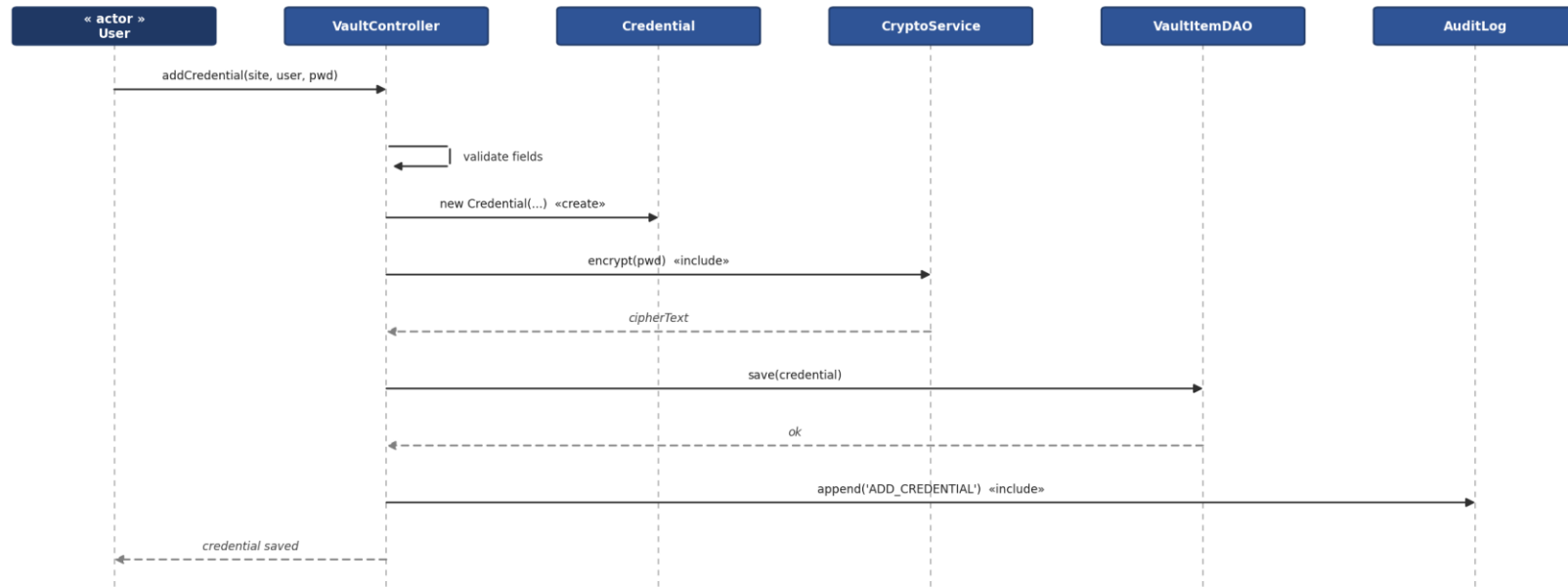


Figure 3. Sequence realization of UC-02 Add Credential.

### ***1.3.3 UC-03 Retrieve Credential***

The record is fetched via the VaultItemDAO, decrypted on demand by the CryptoService, and the access is logged. Exceptions: a missing entry returns not-found; a decryption failure denies access rather than returning corrupt data.

### Sequence Diagram — UC-03 Retrieve Credential

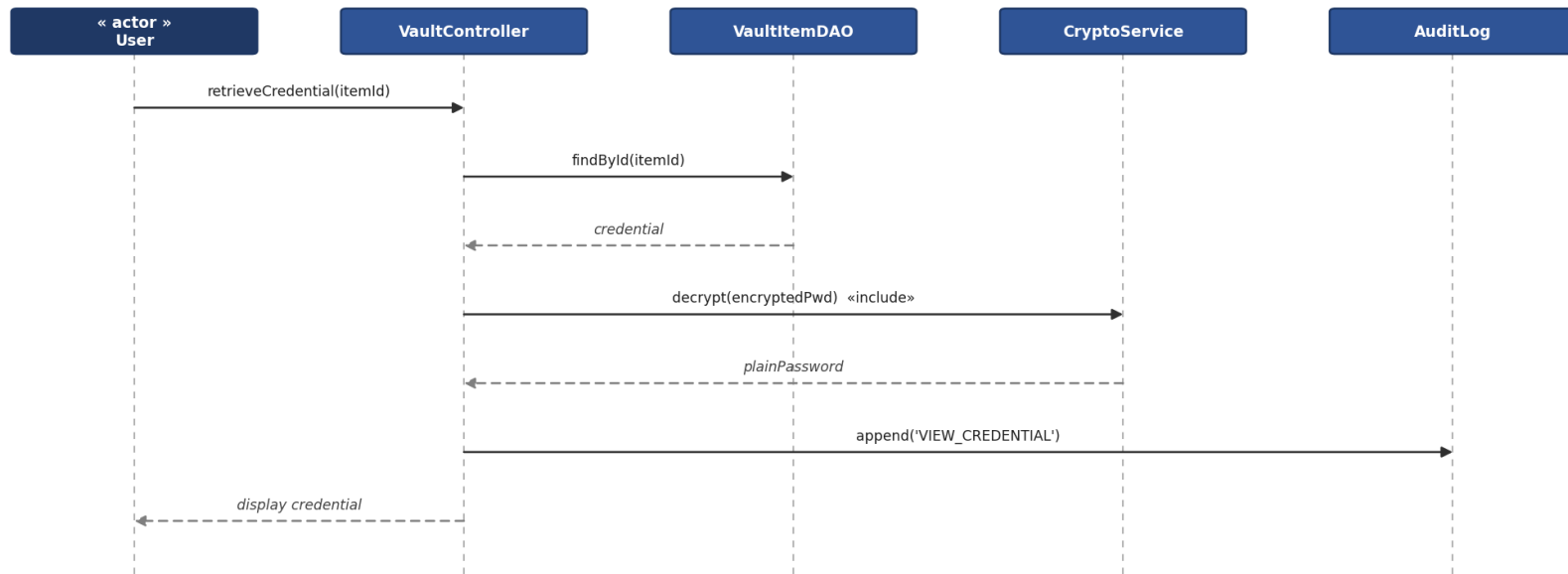


Figure 4. Sequence realization of UC-03 Retrieve Credential.

#### ***1.3.4 UC-04 Store Sensitive Document***

The VaultController checks file type and size, the CryptoService encrypts the document, and the encrypted blob is saved and logged. Exceptions: an unsupported or oversized file is rejected before encryption; an encryption failure prevents storage.

### Sequence Diagram — UC-04 Store Sensitive Document

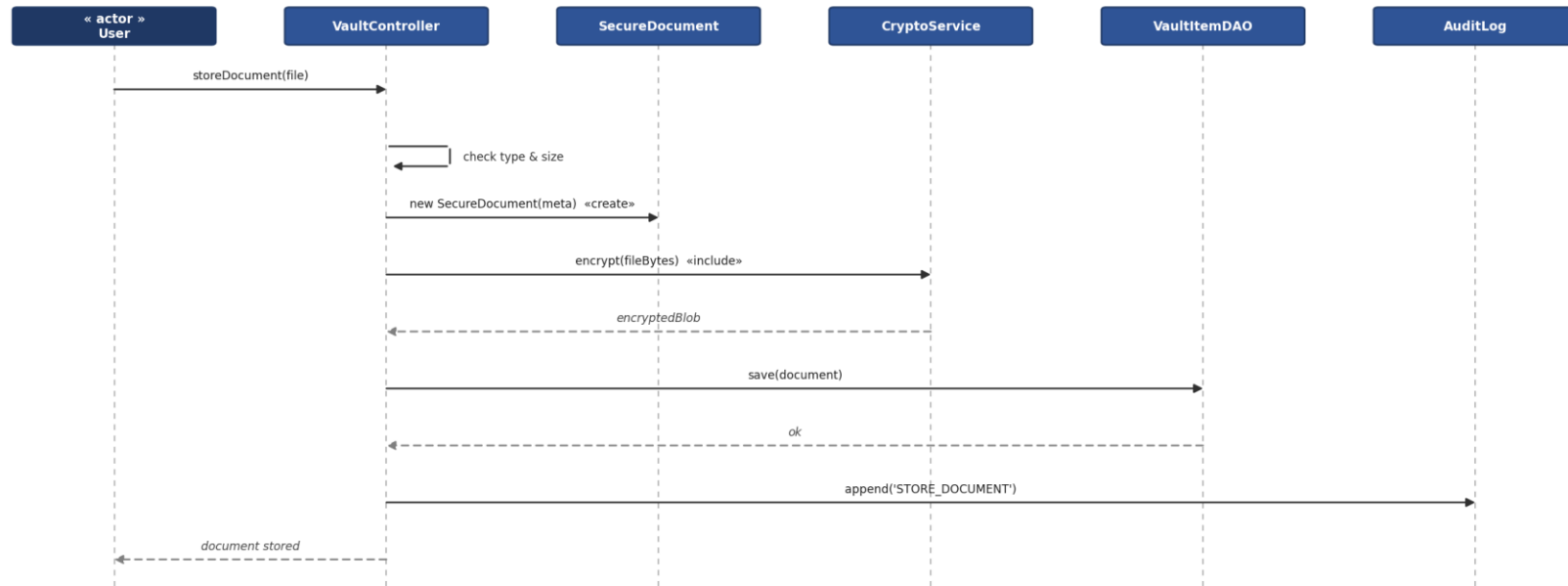


Figure 5. Sequence realization of UC-04 Store Sensitive Document.

#### ***1.3.5 UC-05 Analyze Password Health & Notify***

The PasswordHealthReport reads all credentials via the VaultItemDAO, scores strength, and checks for reuse in the last thirty days; the «extend» Notify User step raises a SecurityAlert per risk and sends it through the external EmailService. Exception: an empty vault yields a report with no entries.



Sequence Diagram — UC-05 Analyze Password Health & Notify

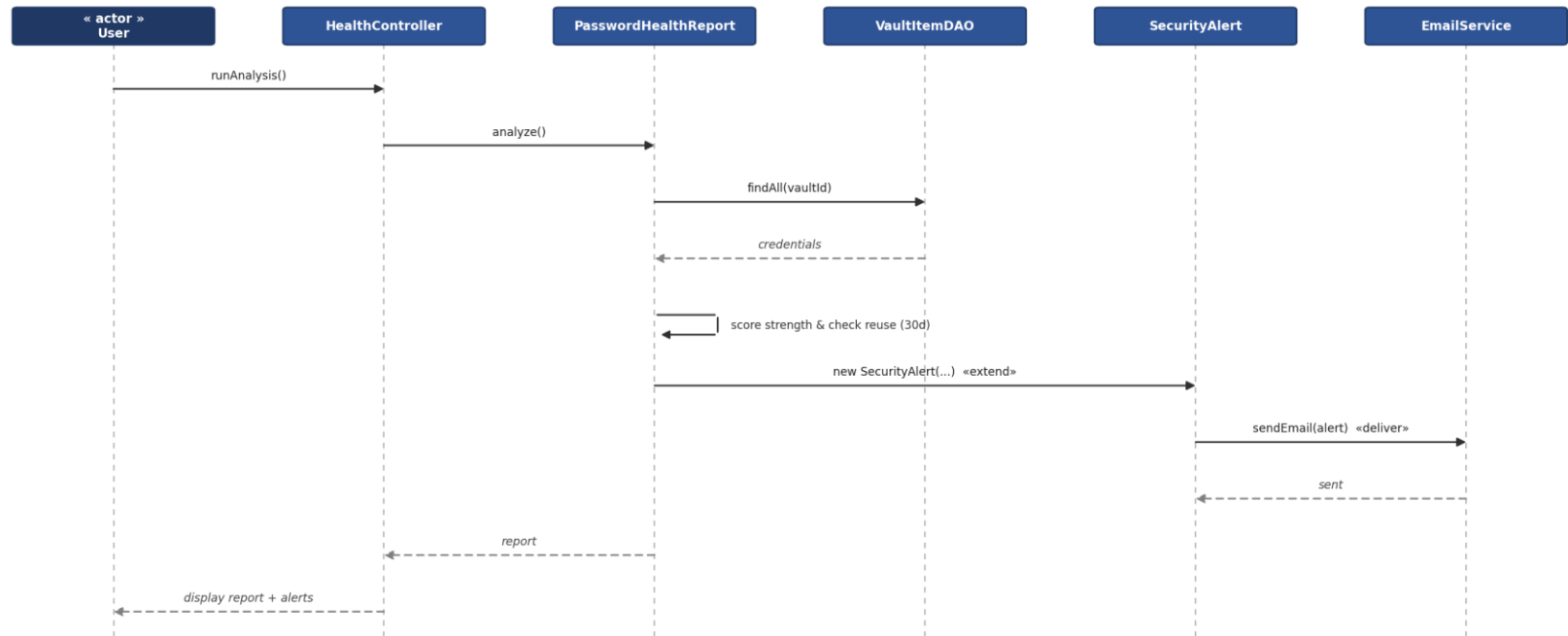


Figure 6. Sequence realization of UC-05 Analyze Password Health & Notify.

## 2. User Interface / User Experience Design

The interface turns the functional design into usable screens. For a security product the challenge is balance: the protections that make SecureVault trustworthy must not make it tedious. The design applies established usability principles plus conventions specific to handling secrets — presented here as principles, a navigation structure, the login-and-2FA flow, and six wireframes.

### 2.1 Design Principles Applied

The interface follows widely used usability heuristics, extended with a secure-by-default principle for vault data. Each principle below maps to a concrete decision in the wireframes.

| Principle                           | Application in SecureVault                                                                                                              |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Visibility of system status         | A persistent auto-lock countdown and encrypting / loading indicators keep the user aware of system state.                               |
| Match to the real world             | Plain vocabulary (“vault”, “credential”, “document”) is used instead of technical jargon.                                               |
| User control and freedom            | A reveal toggle on every password, a Cancel action on every form, and Back navigation from each screen.                                 |
| Consistency and standards           | A shared header, button styles, and iconography are used across all screens.                                                            |
| Error prevention                    | Field validation before save, a live strength meter, and stated file type / size limits stop errors before they happen.                 |
| Recognition over recall             | Inline colour-coded health status, search, and a single headline score remove the need to remember details.                             |
| Flexibility and efficiency          | Flat bottom navigation and an inline generator keep every core task within three clicks.                                                |
| Aesthetic and minimalist design     | Each screen shows only the fields its task requires.                                                                                    |
| Help users recover from errors      | Non-security errors explain the exact cause and fix — for example, a rejected file states the size limit.                               |
| Secure by default (domain-specific) | Secrets are masked by default, copied rather than displayed with a clipboard auto-clear, and irreversible actions require confirmation. |

## 2.2 Navigation Map

After the two-step login, the Vault Dashboard is the hub from which every core task is one tap away and to which each task returns. Two transitions are security-driven (dashed): the generator opens from the credential form, and inactivity beyond ten minutes — or logout — returns to login.

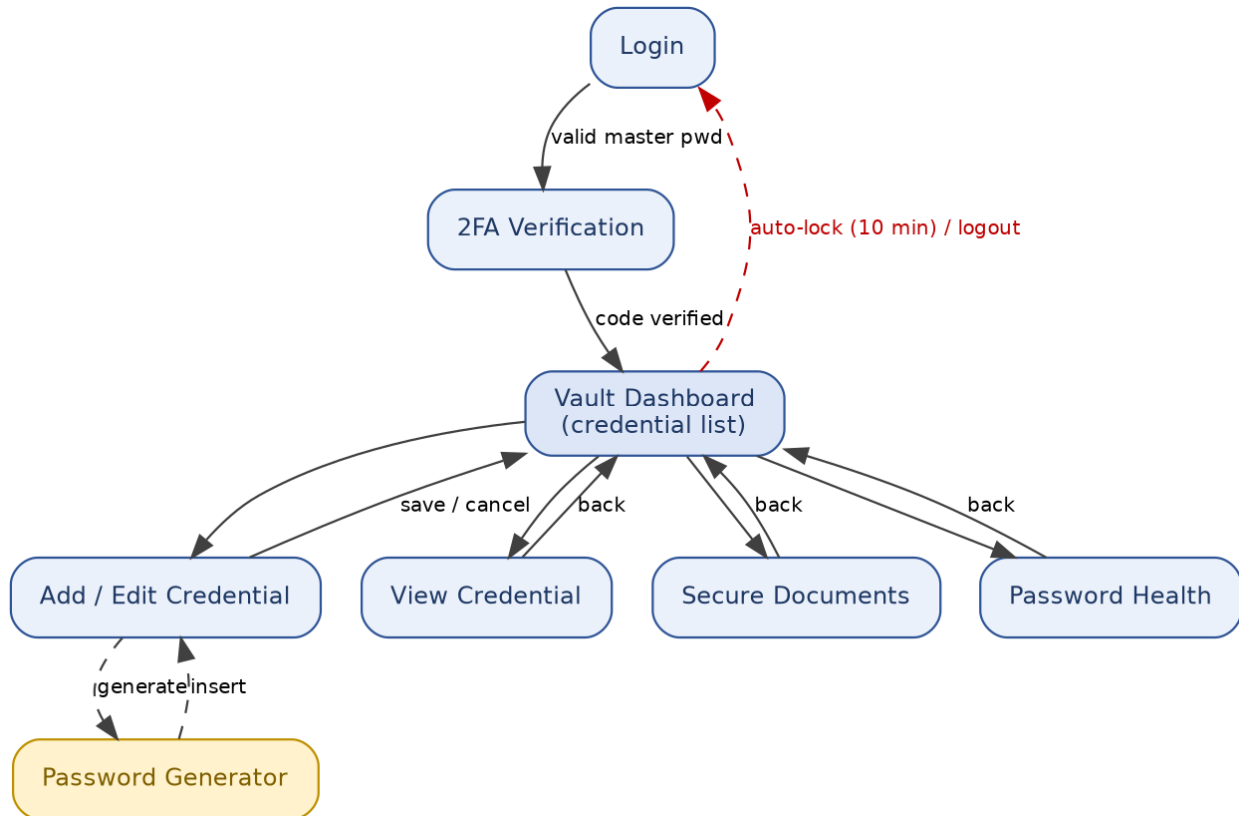


Figure 7. SecureVault screen navigation map.

## 2.3 Login and Two-Factor User Flow

This flow details the most security-critical interaction, including failures. A valid master password advances to the second factor; an invalid password or code increments the attempt counter and, on the fifth failure, locks the account for fifteen minutes. Only after both factors pass does the system create a session, unlock the vault, and log the event.

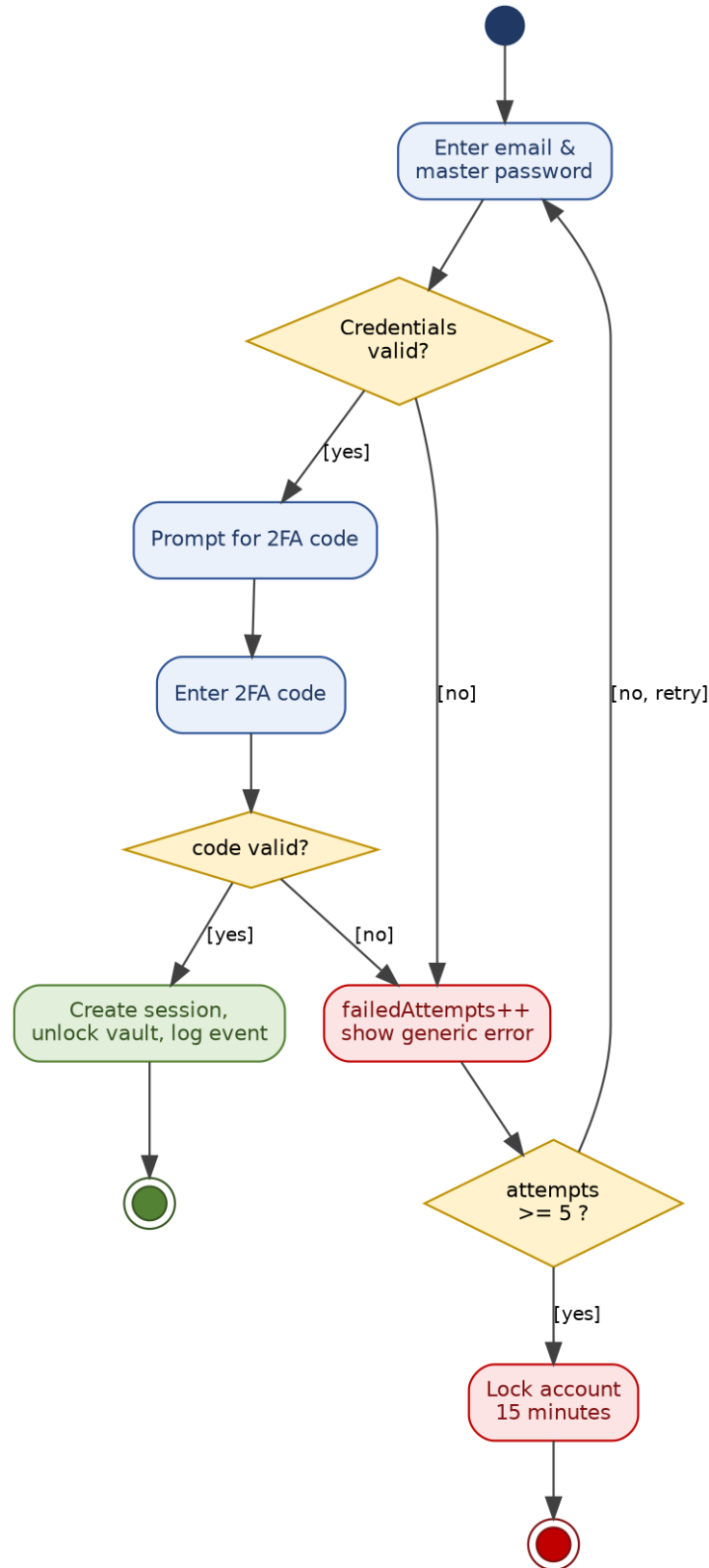


Figure 8. Login and two-factor authentication user flow, including the lockout path.

## 2.4 Screen Wireframes

The six core screens follow, each with numbered callouts explaining the design decisions, summarized beside the screen. Together they cover all five use cases.

### 2.4.1 Login and Two-Factor Verification

SecureVault

Login

Welcome back

Unlock your encrypted vault

Email

Master password

.....

show

Unlock

Step 2 • Two-factor verification

6-digit code

Verify code

Wrong details? Generic "invalid login" message only.

UX / design rationale

1

Password masked by default with an explicit reveal toggle — user control + shoulder-surf protection.

2

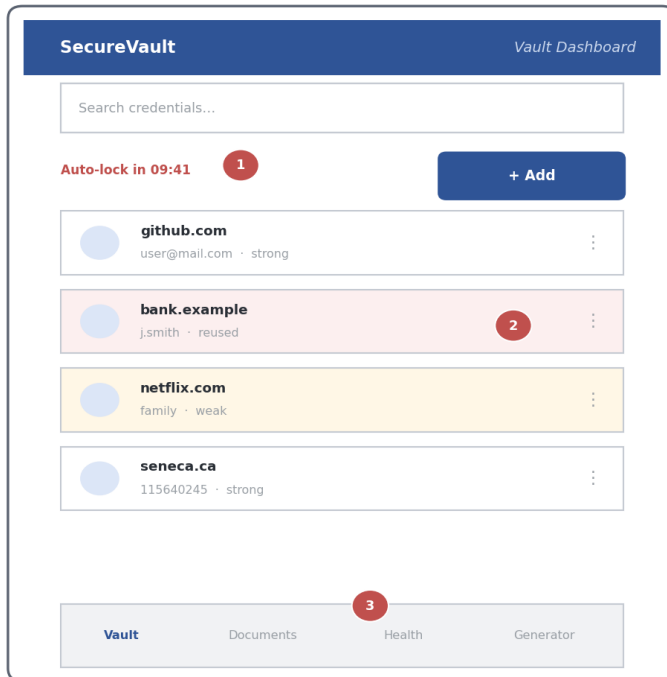
2FA is a separate, clearly numbered step so the login flow is predictable (visibility of system status).

3

Errors never reveal which field was wrong ("invalid login", not "wrong password") to avoid account enumeration.

Figure 9. Login and second-factor verification (UC-01).

## 2.4.2 Vault Dashboard



### UX / design rationale

- 1 Persistent auto-lock countdown gives constant feedback on the 10-minute inactivity rule (business rule made visible).
- 2 Health status is colour-coded inline (reused = red, weak = amber) so risk is recognised, not recalled.
- 3 Flat bottom navigation keeps every core task within one tap of the dashboard (efficiency / few clicks).

Figure 10. The Vault Dashboard — credential list and navigation hub.

## 2.4.3 Add / Edit Credential

The screenshot shows the 'SecureVault' application interface for adding a new credential. The header bar has 'SecureVault' on the left and 'Add Credential' on the right. Below the header is a section titled 'Add credential'. It contains four input fields: 'Website / service' (with 'github.com' entered), 'Username' (with 'user@mail.com' entered), 'Password' (with masked characters '.....' and a 'show' link), and a 'Strength' indicator (a green bar with a red circular callout '1' and the text 'Strength: Strong'). Below the password field is a button labeled 'Generate strong password' with a red circular callout '2'. At the bottom are two buttons: 'Save' and 'Cancel'. A red circular callout '3' points to a red error message at the bottom: 'Empty required fields block Save and are flagged inline.'

### UX / design rationale

- 1 Live strength meter nudges users toward stronger passwords at the point of entry (error prevention).
- 2 One-tap generator reuses the PasswordGenerator class — reduces effort and discourages weak/reused choices.
- 3 Validation happens before save with inline flags, so mistakes are caught early and locally.

Figure 11. Adding or editing a credential (UC-02), with the inline generator and strength meter.

## 2.4.4 View Credential



The interface shows a credential for **github.com** with the text "Last changed 12 days ago". It includes input fields for "Username" (containing "user@mail.com") and "Password" (masked with dots). Below the password field are "Reveal" and "Copy" buttons. A red callout '1' is positioned over the "Copy" button. Below these buttons is a red note: "Clipboard auto-clears after 30 seconds." At the bottom are "Edit" and "Delete" buttons. A red callout '2' is positioned over the "Delete" button. Below the buttons is a yellow confirmation dialog box with a red callout '3' over its title "Delete this credential?". The dialog contains the text "This action cannot be undone."

### UX / design rationale

- 1 Password stays hidden until Reveal; Copy avoids ever displaying it — minimum exposure of secrets.
- 2 Destructive Delete is visually separated from Edit so it isn't triggered by accident.
- 3 Irreversible actions require an explicit confirm dialog (prevent errors; clear consequence).

Figure 12. Viewing a stored credential (UC-03), with reveal, copy, and a delete confirmation.

## 2.4.5 Secure Documents

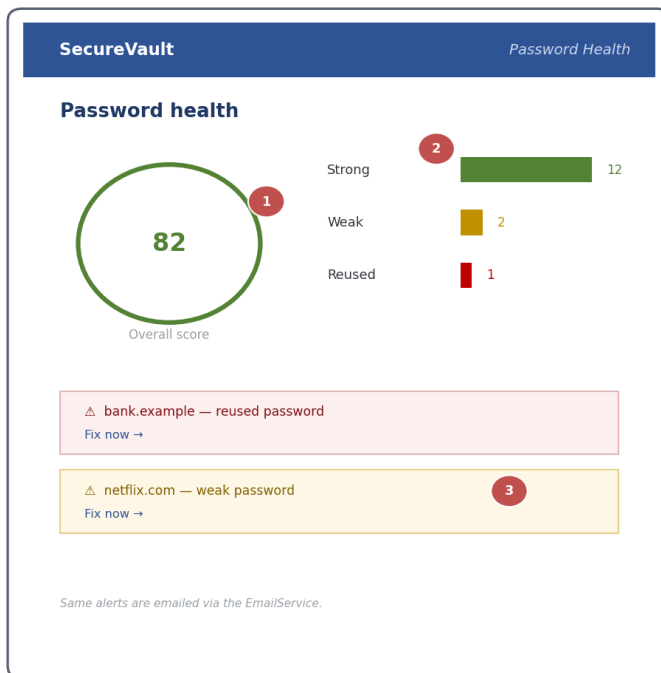


### UX / design rationale

- 1 Drop-zone states the accepted types and size limit up front, so the constraint is known before upload (error prevention).
- 2 An explicit "encrypted" badge reassures the user the zero-knowledge promise is being kept (trust / visibility).
- 3 Rejections explain the exact reason and limit rather than failing silently (helpful error recovery).

Figure 13. Storing a sensitive document (UC-04), with type and size limits stated up front.

## 2.4.6 Password Health



### UX / design rationale

- 1 A single headline score makes overall posture graspable at a glance (recognition over calculation).
- 2 Strong/weak/reused breakdown maps directly to the analyze() rules, so the report is explainable.
- 3 Each risk carries an inline "Fix now" action — the system guides the user to the next step, not just the problem.



Figure 14. The password health report and risk alerts (UC-05).

### 3. Design-to-Requirements Traceability

The table ties the three views together: every screen maps to a use case, every use case is realized by a controller method from the class diagram, and every method persists through the data-access layer.

| Screen                | Use Case                               | Realizing design method(s)                                                      |
|-----------------------|----------------------------------------|---------------------------------------------------------------------------------|
| Login + 2FA           | UC-01 Log In                           | AuthController.handleLogin(), handle2FA()                                       |
| Add / Edit Credential | UC-02 Add Credential                   | VaultController.addCredential() → CryptoService.encrypt() → VaultItemDAO.save() |
| View Credential       | UC-03 Retrieve Credential              | VaultController.retrieveCredential() → CryptoService.decrypt()                  |
| Secure Documents      | UC-04 Store Sensitive Document         | VaultController.storeDocument() → VaultItemDAO.save()                           |
| Password Health       | UC-05 Analyze Password Health & Notify | HealthController.runAnalysis() → PasswordHealthReport.analyze()                 |
| Vault Dashboard       | Navigation hub                         | Vault.search()                                                                  |

### 4. Refinements to Business Requirements

Design surfaced three refinements to the Milestone 3 requirements, as the brief anticipated. First, encryption was extracted into a dedicated CryptoService so the zero-knowledge boundary is explicit and testable. Second, two interface safeguards were added: passwords are copied rather than shown, with the clipboard cleared after thirty seconds, and login errors are generic to prevent account enumeration. Third, master-password checks compare hashes through the CryptoService, never plaintext. None change scope; they make the requirements implementable.

### 5. Summary

This milestone defined how SecureVault will be built. The software design promoted the domain model into a layered design — controllers, a data-access layer, and a cryptographic service — and realized all five use cases as sequence diagrams. The UI/UX design turned the same use cases into a navigation structure, a login-and-2FA flow, and six annotated wireframes.

Because every screen, interaction, and method traces back to a use case, the design is consistent and ready to guide implementation on the Node.js / Express and MySQL stack. The remaining Milestone 4 components — architecture and security, database, and test plans — build on this foundation.

## Part II — Database Architecture

This part maps the SecureVault domain model from Milestone 3 to a normalised relational schema for MySQL 8. It records the database choice, the naming standard applied throughout, the full data dictionary, the entity relationships, the Data Definition Language (DDL) that creates the schema, and the design decisions behind it.

### 1. Database Choice & Naming Conventions

SecureVault uses a relational database (MySQL 8, InnoDB engine, utf8mb4 character set). The data is highly structured and relationship-heavy (users, vaults, items, reports), which suits tables, foreign keys, and joins; InnoDB additionally provides the referential integrity and transactions the design depends on. The schema follows a single, consistent naming standard:

| Object      | Rule                                  | Example                       |
|-------------|---------------------------------------|-------------------------------|
| Tables      | UPPER_SNAKE_CASE, plural              | USERS, VAULT_ITEMS            |
| Columns     | lower_snake_case                      | user_id, master_password_hash |
| Primary key | <entity>_id                           | user_id                       |
| Foreign key | same name as the referenced PK        | user_id in VAULTS             |
| Constraints | PK_ / FK_<child>_<parent> / UQ_ / CK_ | FK_VAULTS_USERS, CK_PHR_SCORE |
| Indexes     | IX_<table>_<cols>                     | IX_AUDIT_USER_TIME            |

Table names are written in uppercase plural form. The singular forms USER and SESSION are SQL reserved words, so the plural convention also avoids the need to quote identifiers. (On case-sensitive platforms, the MySQL setting `lower_case_table_names` should be configured consistently between development and the deployed Cloud SQL instance.)

### 2. Data Dictionary

All tables use the UPPER\_SNAKE\_CASE plural convention; columns use lower\_snake\_case. Secrets are stored only as salted hashes or AES-256-GCM ciphertext with a dedicated nonce (IV) and authentication tag, never as plaintext.

| Table | Column               | Type         | Key | Notes                                                  |
|-------|----------------------|--------------|-----|--------------------------------------------------------|
| USERS | user_id              | INT          | PK  | Auto-increment                                         |
|       | email                | VARCHAR(255) | UQ  | Unique, not null; login identifier                     |
|       | master_password_hash | VARCHAR(255) |     | Argon2id/bcrypt; salt embedded; plaintext never stored |
|       | failed_attempts      | INT          |     | Default 0; brute-force counter                         |
|       | is_locked            | BOOLEAN      |     | Default FALSE                                          |
|       | lockout_until        | DATETIME     |     | NULL; time until which login is blocked (15-min rule)  |
|       | created_at           | DATETIME     |     | Default CURRENT_TIMESTAMP                              |

| Table              | Column             | Type           | Key    | Notes                                             |
|--------------------|--------------------|----------------|--------|---------------------------------------------------|
| VAULTS             | vault_id           | INT            | PK     | Auto-increment                                    |
|                    | user_id            | INT            | FK, UQ | → USERS; unique (1:1)                             |
|                    | auto_lock_minutes  | INT            |        | Default 10; CHECK > 0                             |
|                    | is_locked          | BOOLEAN        |        | Default TRUE                                      |
| VAULT_ITEMS        | item_id            | INT            | PK     | Auto-increment                                    |
|                    | vault_id           | INT            | FK     | → VAULTS                                          |
|                    | item_type          | ENUM           |        | 'CREDENTIAL' / 'DOCUMENT' (discriminator)         |
|                    | title              | VARCHAR(255)   |        | Not null                                          |
|                    | created_at         | DATETIME       |        | Default CURRENT_TIMESTAMP                         |
|                    | updated_at         | DATETIME       |        | Auto-updates on change                            |
| CREDENTIALS        | item_id            | INT            | PK, FK | → VAULT_ITEMS (shared PK, subtype)                |
|                    | url                | VARCHAR(512)   |        | Nullable                                          |
|                    | username           | VARCHAR(255)   |        | Nullable                                          |
|                    | encrypted_password | VARBINARY(512) |        | AES-256-GCM ciphertext                            |
|                    | password_iv        | VARBINARY(12)  |        | GCM nonce                                         |
|                    | password_tag       | VARBINARY(16)  |        | GCM authentication tag                            |
| SECURE_DOCUMENTS   | last_changed       | DATETIME       |        | Default CURRENT_TIMESTAMP                         |
|                    | item_id            | INT            | PK, FK | → VAULT_ITEMS (shared PK, subtype)                |
|                    | file_name          | VARCHAR(255)   |        | Not null                                          |
|                    | file_type          | VARCHAR(50)    |        | CHECK in (application/pdf, image/png, image/jpeg) |
|                    | file_size_kb       | INT            |        | CHECK 1-10240 (10 MB rule)                        |
|                    | encrypted_blob     | LONGBLOB       |        | AES-256-GCM ciphertext                            |
|                    | file_iv            | VARBINARY(12)  |        | GCM nonce                                         |
|                    | file_tag           | VARBINARY(16)  |        | GCM authentication tag                            |
| TWO_FACTOR_CONFIGS | tfa_id             | INT            | PK     | Auto-increment                                    |
|                    | user_id            | INT            | FK, UQ | → USERS; unique                                   |
|                    | method             | ENUM           |        | 'TOTP' / 'EMAIL'                                  |
|                    | secret_enc         | VARBINARY(255) |        | Encrypted 2FA secret                              |
|                    | secret_iv          | VARBINARY(12)  |        | GCM nonce                                         |
|                    | secret_tag         | VARBINARY(16)  |        | GCM authentication tag                            |
|                    | enabled            | BOOLEAN        |        | Default FALSE                                     |

| Table                   | Column        | Type          | Key     | Notes                                                            |
|-------------------------|---------------|---------------|---------|------------------------------------------------------------------|
| SESSIONS                | session_id    | INT           | PK      | Auto-increment                                                   |
|                         | user_id       | INT           | FK      | → USERS                                                          |
|                         | token_hash    | VARBINARY(32) | UQ (IX) | SHA-256 of token; raw token never stored                         |
|                         | started_at    | DATETIME      |         | Default CURRENT_TIMESTAMP                                        |
|                         | expires_at    | DATETIME      |         | Drives idle auto-lock                                            |
| AUDIT_ENTRIES           | entry_id      | INT           | PK      | Auto-increment                                                   |
|                         | user_id       | INT           | FK      | → USERS                                                          |
|                         | action        | VARCHAR(100)  |         | Not null                                                         |
|                         | event_time    | DATETIME      |         | Default CURRENT_TIMESTAMP (renamed from reserved word timestamp) |
|                         | ip_address    | VARCHAR(45)   |         | IPv6-capable; append-only                                        |
| PASSWORD_HEALTH_REPORTS | report_id     | INT           | PK      | Auto-increment                                                   |
|                         | vault_id      | INT           | FK      | → VAULTS                                                         |
|                         | generated_at  | DATETIME      |         | Default CURRENT_TIMESTAMP                                        |
|                         | overall_score | INT           |         | CHECK 0-100                                                      |
| REPORT_FINDINGS         | report_id     | INT           | PK, FK  | → PASSWORD_HEALTH_REPORTS (composite PK)                         |
|                         | item_id       | INT           | PK, FK  | → CREDENTIALS (composite PK)                                     |
|                         | status        | ENUM          |         | 'WEAK' / 'REUSED' / 'OK'                                         |
| SECURITY_ALERTS         | alert_id      | INT           | PK      | Auto-increment                                                   |
|                         | report_id     | INT           | FK      | → PASSWORD_HEALTH_REPORTS                                        |
|                         | type          | ENUM          |         | 'WEAK' / 'REUSED'                                                |
|                         | message       | VARCHAR(255)  |         | Not null                                                         |
|                         | is_read       | BOOLEAN       |         | Default FALSE                                                    |
|                         | created_at    | DATETIME      |         | Default CURRENT_TIMESTAMP                                        |

### 3. Entity Relationships & Cardinality

Figure 1 shows the SecureVault entity-relationship diagram using crow's-foot notation, with the standardized uppercase table names, primary/foreign-key markers, and cardinalities. The relationships and multiplicities are as follows:

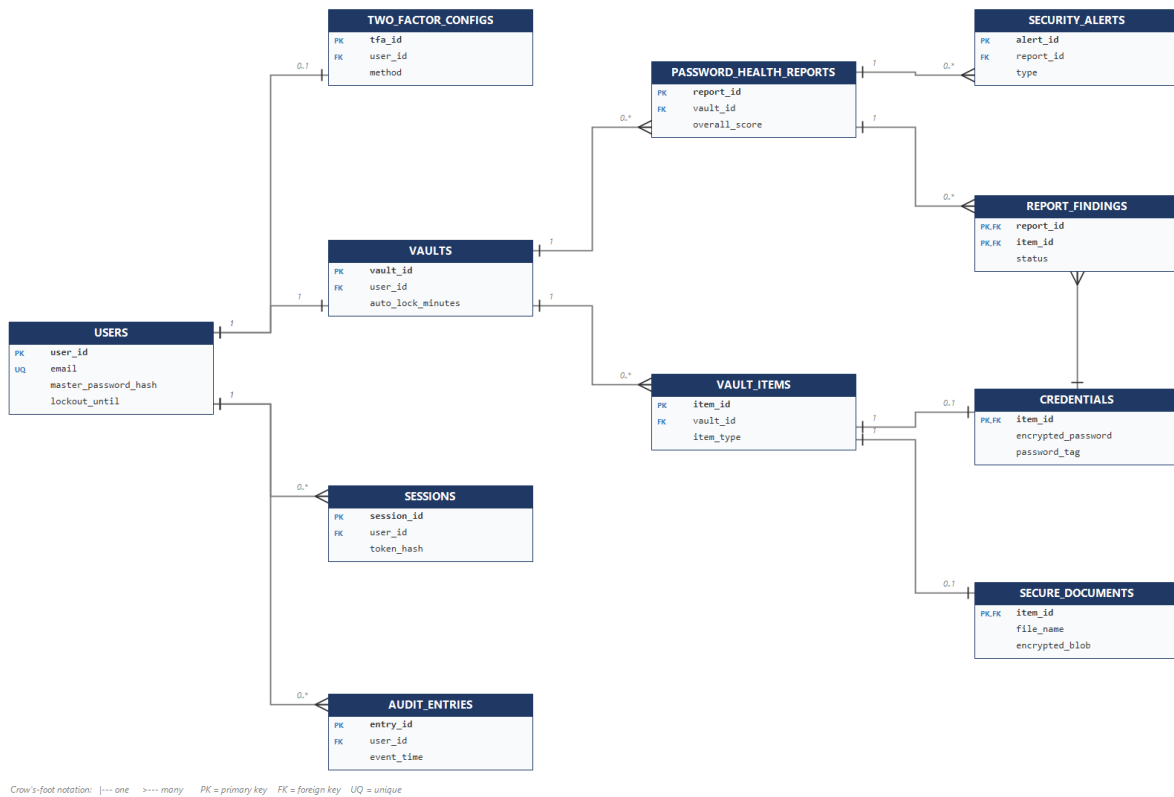


Figure 1. SecureVault Entity-Relationship Diagram (crow's-foot notation).

- ❑ USERS : VAULTS = 1 : 1 — each user owns exactly one vault (enforced by UQ\_VAULTS\_USER).
- ❑ VAULTS : VAULT\_ITEMS = 1 : 0..\* — a vault holds many credentials and documents.
- ❑ VAULT\_ITEMS : CREDENTIALS = 1 : 0..1 and VAULT\_ITEMS : SECURE\_DOCUMENTS = 1 : 0..1 — each item is exactly one subtype (shared primary key).
- ❑ USERS : TWO\_FACTOR\_CONFIGS = 1 : 0..1 ; USERS : SESSIONS = 1 : 0..\* ; USERS : AUDIT\_ENTRIES = 1 : 0..\*.
- ❑ VAULTS : PASSWORD\_HEALTH\_REPORTS = 1 : 0..\*.
- ❑ PASSWORD\_HEALTH\_REPORTS : CREDENTIALS = many-to-many, resolved by the REPORT\_FINDINGS junction table (composite primary key).
- ❑ PASSWORD\_HEALTH\_REPORTS : SECURITY\_ALERTS = 1 : 0..\*.

## 4. Data Definition Language (DDL)

The following MySQL 8 script creates the schema exactly as specified above, including primary and foreign keys, unique and check constraints, cascade rules, and supporting indexes.

```
-- SecureVault relational schema (MySQL 8, InnoDB, utf8mb4)
-- Naming: TABLES = UPPER_SNAKE_CASE (plural); columns = lower_snake_case;
-- constraints = PK_/FK_/UQ_/CK_ ; indexes = IX_

CREATE DATABASE IF NOT EXISTS SECUREVAULT
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE SECUREVAULT;
```

```

-- 1. USERS
CREATE TABLE USERS (
  user_id          INT          NOT NULL AUTO_INCREMENT,
  email            VARCHAR(255) NOT NULL,
  master_password_hash VARCHAR(255) NOT NULL,          -- Argon2id/bcrypt (salt embedded)
  failed_attempts  INT          NOT NULL DEFAULT 0,
  is_locked        BOOLEAN      NOT NULL DEFAULT FALSE,
  lockout_until    DATETIME     NULL,                  -- drives 15-min auto-unlock
  created_at       DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT PK_USERS PRIMARY KEY (user_id),
  CONSTRAINT UQ_USERS_EMAIL UNIQUE (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 2. VAULTS (1:1 with USERS)
CREATE TABLE VAULTS (
  vault_id         INT          NOT NULL AUTO_INCREMENT,
  user_id          INT          NOT NULL,
  auto_lock_minutes INT          NOT NULL DEFAULT 10,
  is_locked        BOOLEAN      NOT NULL DEFAULT TRUE,
  CONSTRAINT PK_VAULTS PRIMARY KEY (vault_id),
  CONSTRAINT UQ_VAULTS_USER UNIQUE (user_id),
  CONSTRAINT FK_VAULTS_USERS FOREIGN KEY (user_id)
    REFERENCES USERS(user_id) ON DELETE CASCADE,
  CONSTRAINT CK_VAULTS_LOCKMIN CHECK (auto_lock_minutes > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 3. VAULT_ITEMS (supertype; class-table inheritance)
CREATE TABLE VAULT_ITEMS (
  item_id          INT          NOT NULL AUTO_INCREMENT,
  vault_id         INT          NOT NULL,
  item_type        ENUM('CREDENTIAL','DOCUMENT') NOT NULL,  -- discriminator
  title            VARCHAR(255) NOT NULL,
  created_at       DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at       DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT PK_VAULT_ITEMS PRIMARY KEY (item_id),
  CONSTRAINT FK_VAULT_ITEMS_VAULTS FOREIGN KEY (vault_id)
    REFERENCES VAULTS(vault_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 4. CREDENTIALS (subtype, shared PK)
CREATE TABLE CREDENTIALS (
  item_id          INT          NOT NULL,
  url              VARCHAR(512) NULL,
  username         VARCHAR(255) NULL,
  encrypted_password VARBINARY(512) NOT NULL,             -- AES-256-GCM ciphertext
  password_iv      VARBINARY(12) NOT NULL,                -- GCM nonce
  password_tag     VARBINARY(16) NOT NULL,                -- GCM auth tag
  last_changed     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT PK_CREDENTIALS PRIMARY KEY (item_id),
  CONSTRAINT FK_CREDENTIALS_ITEMS FOREIGN KEY (item_id)
    REFERENCES VAULT_ITEMS(item_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 5. SECURE_DOCUMENTS (subtype, shared PK)
CREATE TABLE SECURE_DOCUMENTS (
  item_id          INT          NOT NULL,
  file_name        VARCHAR(255) NOT NULL,
  file_type        VARCHAR(50)  NOT NULL,
  file_size_kb     INT          NOT NULL,
  encrypted_blob   LONGBLOB     NOT NULL,                -- AES-256-GCM ciphertext
  file_iv          VARBINARY(12) NOT NULL,
  file_tag         VARBINARY(16) NOT NULL,
  CONSTRAINT PK_SECURE_DOCUMENTS PRIMARY KEY (item_id),
  CONSTRAINT FK_SECURE_DOCUMENTS_ITEMS FOREIGN KEY (item_id)
    REFERENCES VAULT_ITEMS(item_id) ON DELETE CASCADE,

```

```

CONSTRAINT CK_SECURE_DOCUMENTS_SIZE CHECK (file_size_kb BETWEEN 1 AND 10240),
CONSTRAINT CK_SECURE_DOCUMENTS_TYPE CHECK (file_type IN
('application/pdf','image/png','image/jpeg'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 6. TWO_FACTOR_CONFIGS (0..1 per user)
CREATE TABLE TWO_FACTOR_CONFIGS (
  tfa_id INT NOT NULL AUTO_INCREMENT,
  user_id INT NOT NULL,
  method ENUM('TOTP','EMAIL') NOT NULL,
  secret_enc VARBINARY(255) NOT NULL, -- encrypted secret
  secret_iv VARBINARY(12) NOT NULL,
  secret_tag VARBINARY(16) NOT NULL,
  enabled BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT PK_TWO_FACTOR_CONFIGS PRIMARY KEY (tfa_id),
  CONSTRAINT UQ_TFA_USER UNIQUE (user_id),
  CONSTRAINT FK_TFA_USERS FOREIGN KEY (user_id)
  REFERENCES USERS(user_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 7. SESSIONS
CREATE TABLE SESSIONS (
  session_id INT NOT NULL AUTO_INCREMENT,
  user_id INT NOT NULL,
  token_hash VARBINARY(32) NOT NULL, -- SHA-256 of token; raw token never stored
  started_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  expires_at DATETIME NOT NULL, -- drives idle auto-lock
  CONSTRAINT PK_SESSIONS PRIMARY KEY (session_id),
  CONSTRAINT FK_SESSIONS_USERS FOREIGN KEY (user_id)
  REFERENCES USERS(user_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
CREATE UNIQUE INDEX IX_SESSIONS_TOKEN_HASH ON SESSIONS(token_hash);
CREATE INDEX IX_SESSIONS_EXPIRES ON SESSIONS(expires_at);

-- 8. AUDIT_ENTRIES (append-only)
CREATE TABLE AUDIT_ENTRIES (
  entry_id INT NOT NULL AUTO_INCREMENT,
  user_id INT NOT NULL,
  action VARCHAR(100) NOT NULL,
  event_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, -- was reserved word "timestamp"
  ip_address VARCHAR(45) NULL, -- IPv6-capable
  CONSTRAINT PK_AUDIT_ENTRIES PRIMARY KEY (entry_id),
  CONSTRAINT FK_AUDIT_USERS FOREIGN KEY (user_id)
  REFERENCES USERS(user_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
CREATE INDEX IX_AUDIT_USER_TIME ON AUDIT_ENTRIES(user_id, event_time);
-- Append-only is enforced by granting the app role only INSERT/SELECT on this table.

-- 9. PASSWORD_HEALTH_REPORTS
CREATE TABLE PASSWORD_HEALTH_REPORTS (
  report_id INT NOT NULL AUTO_INCREMENT,
  vault_id INT NOT NULL,
  generated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  overall_score INT NOT NULL,
  CONSTRAINT PK_PHR PRIMARY KEY (report_id),
  CONSTRAINT FK_PHR_VAULTS FOREIGN KEY (vault_id)
  REFERENCES VAULTS(vault_id) ON DELETE CASCADE,
  CONSTRAINT CK_PHR_SCORE CHECK (overall_score BETWEEN 0 AND 100)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 10. REPORT_FINDINGS (junction, composite PK)
CREATE TABLE REPORT_FINDINGS (
  report_id INT NOT NULL,
  item_id INT NOT NULL,
  status ENUM('WEAK','REUSED','OK') NOT NULL,
  CONSTRAINT PK_REPORT_FINDINGS PRIMARY KEY (report_id, item_id),

```

```

CONSTRAINT FK_RF_REPORTS FOREIGN KEY (report_id)
REFERENCES PASSWORD_HEALTH_REPORTS(report_id) ON DELETE CASCADE,
CONSTRAINT FK_RF_CREDENTIALS FOREIGN KEY (item_id)
REFERENCES CREDENTIALS(item_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 11. SECURITY_ALERTS
CREATE TABLE SECURITY_ALERTS (
  alert_id INT NOT NULL AUTO_INCREMENT,
  report_id INT NOT NULL,
  type ENUM('WEAK', 'REUSED') NOT NULL,
  message VARCHAR(255) NOT NULL,
  is_read BOOLEAN NOT NULL DEFAULT FALSE,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT PK_SECURITY_ALERTS PRIMARY KEY (alert_id),
  CONSTRAINT FK_ALERTS_REPORTS FOREIGN KEY (report_id)
REFERENCES PASSWORD_HEALTH_REPORTS(report_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

## 5. Design Notes

- ❑ Inheritance mapping: VaultItem uses class-table inheritance — a base VAULT\_ITEMS table plus CREDENTIALS and SECURE\_DOCUMENTS subtype tables sharing the same primary key. item\_type is the discriminator, which avoids null-heavy columns.
- ❑ Composition / cascades: foreign keys use ON DELETE CASCADE, so deleting a user removes its vault, items, sessions, 2FA config, and audit entries — matching the rule that a part cannot exist without its whole.
- ❑ Many-to-many: the report-to-credential link is resolved with the REPORT\_FINDINGS junction table using a composite primary key.
- ❑ Normalization: all tables are in third normal form (3NF); no repeating groups or transitive dependencies.
- ❑ Security: the master password is stored only as a salted hash; credential passwords, document blobs, and 2FA secrets are AES-256-GCM ciphertext (each with its own IV and authentication tag), never plaintext; session tokens are stored only as SHA-256 hashes; TLS protects all data in transit.
- ❑ Indexing: a unique index on USERS.email, a unique index on SESSIONS.token\_hash, an index on SESSIONS.expires\_at for expiry sweeps, a composite index on AUDIT\_ENTRIES(user\_id, event\_time), and indexes on every foreign-key column.
- ❑ Refinements from the design recheck: added logout\_until to support the 15-minute logout; added GCM nonce/tag columns for every encrypted field; store session tokens as hashes rather than raw values; renamed the audit time column off the reserved word timestamp; and added CHECK constraints for score range, file size, and file type.
- ❑ External services: AuthenticationService and EmailService are not stored in the database; the application calls them at runtime.



## Part III — System Architecture & Security: Cloud Deployment & DevOps

### 1. Introduction & Executive Summary

This Milestone 4 report presents the system design for the deployment of SecureVault — the Secure Password Management System. Where earlier milestones defined the project plan (Milestone 2) and the requirements analysis (Milestone 3), this document specifies how the application is packaged, provisioned, deployed, operated, and torn down in a live cloud environment.

The deployment rests on three pillars: Google Cloud Platform (GCP) as the hosting environment, Terraform as the infrastructure-as-code (IaC) tool that provisions every resource, and GitHub Actions as the continuous integration and delivery (CI/CD) engine that builds, tests, and ships each change. All source and infrastructure code lives in the team repository, SPMS\_Deployment.

The design is deliberately cost-aware. The full environment is sized to run comfortably within the \$300 GCP free-trial credit across a two-month academic window, and the entire stack can be destroyed with a single command once grading is complete, returning spend to zero.

The sections that follow cover the technology stack, the deployment architecture and its diagrams, the Terraform-provisioned resources, the CI/CD pipeline and deployment strategy, an estimated cost breakdown, the rationale behind each major decision, and a set of operational considerations covering secrets, security, backups, and budget.

**Scope.** This milestone covers the deployment and DevOps view of SecureVault. The application-level design (class model, sequence flows, and UI/UX) is documented in the team's parallel design deliverable; this document treats the application as a containerised Node.js/Express service and focuses on how it is run.

### 2. Technology Stack

The deployment combines a small, well-supported set of tools. Each was chosen to minimise operational overhead and stay within free or low-cost tiers.

| Layer                  | Technology               | Role in SecureVault                                                                  |
|------------------------|--------------------------|--------------------------------------------------------------------------------------|
| Source control         | GitHub (SPMS_Deployment) | Single source of truth for application and infrastructure code.                      |
| CI/CD                  | GitHub Actions           | Automated build, test, and deploy on every push and pull request.                    |
| Infrastructure as Code | Terraform                | Declarative provisioning, versioning, and one-command teardown of all GCP resources. |
| Cloud platform         | Google Cloud Platform    | Hosting environment; \$300 free-trial credit available.                              |
| Containerisation       | Docker                   | Packages the Node.js/Express app into a                                              |

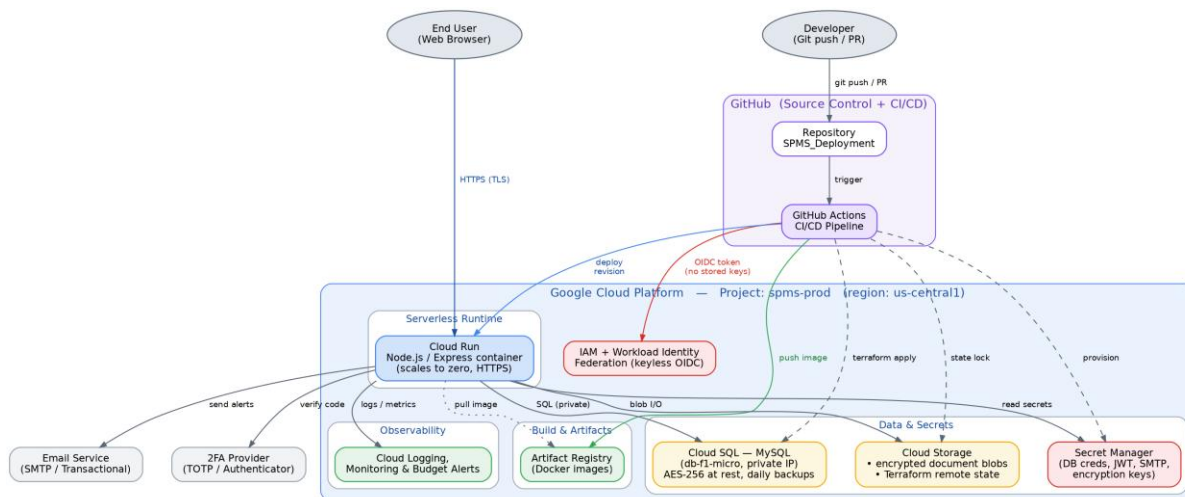
| Layer             | Technology                         | Role in SecureVault                                                   |
|-------------------|------------------------------------|-----------------------------------------------------------------------|
|                   |                                    | portable, immutable image.                                            |
| Image registry    | Artifact Registry                  | Stores built Docker images for deployment to Cloud Run.               |
| Compute           | Cloud Run                          | Serverless container runtime; scales to zero when idle.               |
| Database          | Cloud SQL for MySQL                | Managed relational store for vault and user data.                     |
| Object storage    | Cloud Storage                      | Encrypted document blobs and the Terraform remote state.              |
| Secrets           | Secret Manager                     | Stores DB credentials, JWT and encryption keys, and SMTP credentials. |
| Identity & access | Workload Identity Federation + IAM | Keyless, least-privilege access for the pipeline and runtime.         |
| Observability     | Cloud Logging & Monitoring         | Application logs, metrics, uptime checks, and budget alerts.          |

The application stack itself is carried over from prior milestones: Node.js with Express, a MySQL database, AES-256 encryption at rest, TLS in transit, and two-factor authentication. The deployment layer is largely engine-agnostic but is tuned to this stack.

### 3. Deployment Architecture

#### 3.1 Architecture Overview

The architecture is organised into three zones: GitHub, which holds the code and runs the CI/CD pipeline; the GCP project that hosts every runtime resource; and the external actors and services that interact with the system. Figure 3.1 shows the full topology, including both the build/deploy path (top) and the runtime data path (bottom).



*Figure 3.1: SecureVault GCP deployment architecture — resources, CI/CD path, and runtime connections.*

On the deploy path, a developer pushes to GitHub, which triggers GitHub Actions. The pipeline authenticates to GCP without any stored keys using Workload Identity Federation, pushes the built image to Artifact Registry, runs Terraform to converge infrastructure, and deploys a new Cloud Run revision. On the runtime path, end users reach Cloud Run over HTTPS; the service reads secrets from Secret Manager, queries Cloud SQL over a private path, stores document blobs in Cloud Storage, and emits logs and metrics to Cloud Operations.

### 3.2 Component Responsibilities

| Component                | Responsibility                                                              | Key configuration                                |
|--------------------------|-----------------------------------------------------------------------------|--------------------------------------------------|
| Cloud Run                | Hosts the containerised Node.js/Express app; terminates HTTPS; auto-scales. | 1 vCPU / 512 MiB, min=0, max=2                   |
| Cloud SQL (MySQL)        | Managed relational store for users, credentials, and audit data.            | db-f1-micro, 10 GB SSD, private IP, MySQL 8.4    |
| Cloud Storage            | Encrypted document blobs and versioned Terraform state.                     | Two buckets, uniform access, versioning on state |
| Secret Manager           | Central store for all runtime secrets, injected at start-up.                | DB creds, JWT key, AES key, SMTP creds           |
| Artifact Registry        | Stores immutable Docker images tagged by git commit SHA.                    | Docker format, single regional repo              |
| IAM + WIF                | Identity for the pipeline and runtime; least-privilege roles.               | Keyless OIDC federation, scoped SAs              |
| VPC (Direct egress)      | Private network path from Cloud Run to Cloud SQL.                           | Direct VPC egress (scales to zero)               |
| Cloud Logging/Monitoring | Centralised logs, metrics, uptime checks, budget alerts.                    | Default sink + budget at \$300                   |

### 3.3 Runtime Request Flow

Figure 3.2 traces a single authenticated request end to end. The Google Front End terminates TLS and forwards the request to a Cloud Run instance, which fetches secrets on cold start, reads or writes the database, performs blob I/O when documents are involved, and calls external services for 2FA verification and alert e-mails before returning the response.

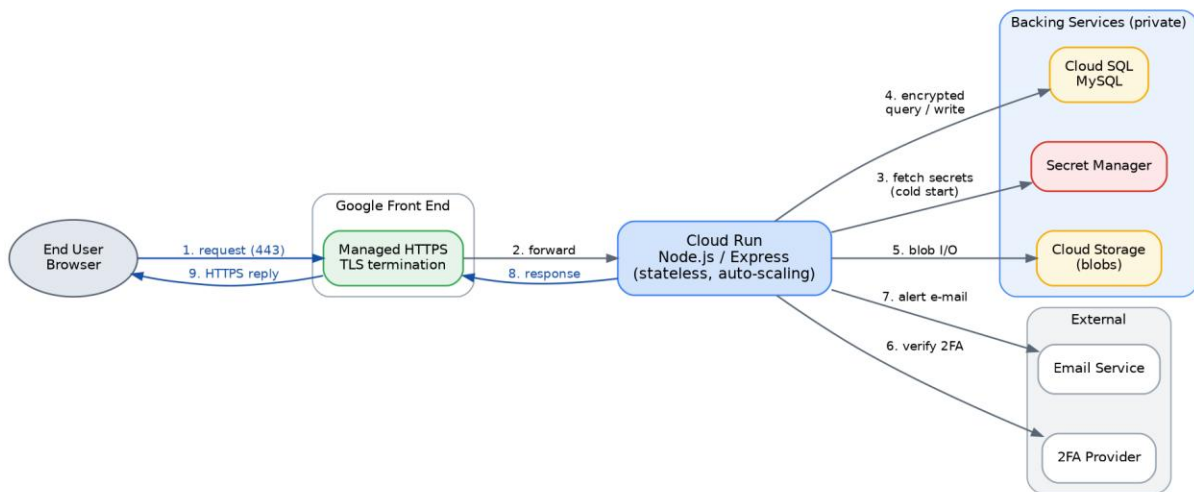


Figure 3.2: SecureVault runtime request flow — from browser through Cloud Run to backing and external services.

### 3.4 Network & Security Boundaries

- ☐ The database is never exposed publicly: Cloud SQL uses a private IP and is reached only over the VPC via Direct VPC egress.
- ☐ All client traffic is HTTPS only; the Google-managed certificate terminates at the Cloud Run front end.
- ☐ Secrets never live in source code or in the Docker image; they are read from Secret Manager at runtime under a dedicated service account.
- ☐ Pipeline and runtime each use their own least-privilege service account, so a compromise of one does not grant the other's access.
- ☐ The design preserves SecureVault's zero-knowledge posture: encryption is applied at the application layer and at rest, so the infrastructure never handles plaintext vault contents.

## 4. Provisioned Resources (Terraform)

Every resource below is declared in Terraform and created, updated, or destroyed through the pipeline. Nothing is clicked together by hand in the console, which keeps the environment reproducible and easy to tear down.

### 4.1 Resource Inventory

| Resource        | GCP service | Terraform resource           | Tier / config                      |
|-----------------|-------------|------------------------------|------------------------------------|
| App service     | Cloud Run   | google_cloud_run_v2_service  | 1 vCPU / 512 MiB, min 0 / max 2    |
| MySQL instance  | Cloud SQL   | google_sql_database_instance | db-f1-micro, 10 GB SSD, private IP |
| Database & user | Cloud SQL   | google_sql_database / _user  | App schema and login               |

| Resource         | GCP service       | Terraform resource                           | Tier / config                  |
|------------------|-------------------|----------------------------------------------|--------------------------------|
| Image repository | Artifact Registry | google_artifact_registry_repository          | Docker format, regional        |
| State bucket     | Cloud Storage     | google_storage_bucket                        | Versioned, uniform access      |
| Document bucket  | Cloud Storage     | google_storage_bucket                        | Standard, lifecycle rules      |
| Secrets          | Secret Manager    | google_secret_manager_secret(_version)       | ~6 secrets                     |
| Runtime identity | IAM               | google_service_account                       | Least-privilege Cloud Run SA   |
| Deployer + WIF   | IAM               | google_iam_workload_identity_pool(_provider) | Keyless CI/CD identity         |
| Role bindings    | IAM               | google_project_iam_member                    | Scoped roles only              |
| Private network  | VPC               | google_compute_network / _subnetwork         | Direct VPC egress to Cloud SQL |
| Budget alert     | Billing           | google_billing_budget                        | \$300 cap; 50/90/100% alerts   |

## 4.2 Terraform Project Structure

The Terraform code is split into small, single-purpose modules so that resources can be reasoned about and changed independently. A thin root configuration wires the modules together and selects the remote state backend.

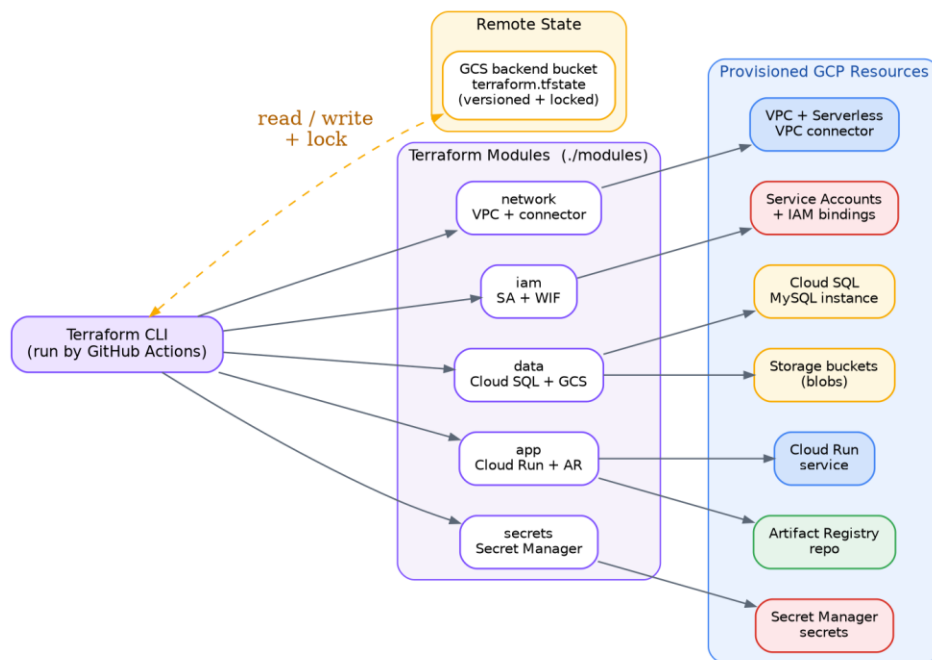


Figure 4.1: Terraform provisioning model — CLI, remote state, modules, and the GCP resources they create.

Repository layout:



### 4.3 Remote State & Locking

Terraform state is stored in a dedicated, versioned Cloud Storage bucket rather than on a developer's laptop. The GCS backend provides built-in state locking, which prevents two pipeline runs from corrupting state by writing at the same time. Versioning on the bucket means any bad apply can be rolled back to a previous state file.

### 4.4 Cost-Aware Sizing

Several choices keep the environment small. Cloud Run runs with min-instances of zero so it costs nothing while idle; Direct VPC egress is used instead of a Serverless VPC connector so there is no always-on connector instance to pay for; Cloud SQL uses the shared-core db-f1-micro machine; and all resources sit in a single Tier-1 region so that data transfer between them is free.

## 5. CI/CD Pipeline (GitHub Actions)

### 5.1 Pipeline Overview

Two workflows drive delivery. A CI workflow runs on every pull request and must pass before code can merge; a CD workflow runs after a merge to main and carries the change through to a live Cloud Run revision. Figure 5.1 shows both lanes and the rollback path.

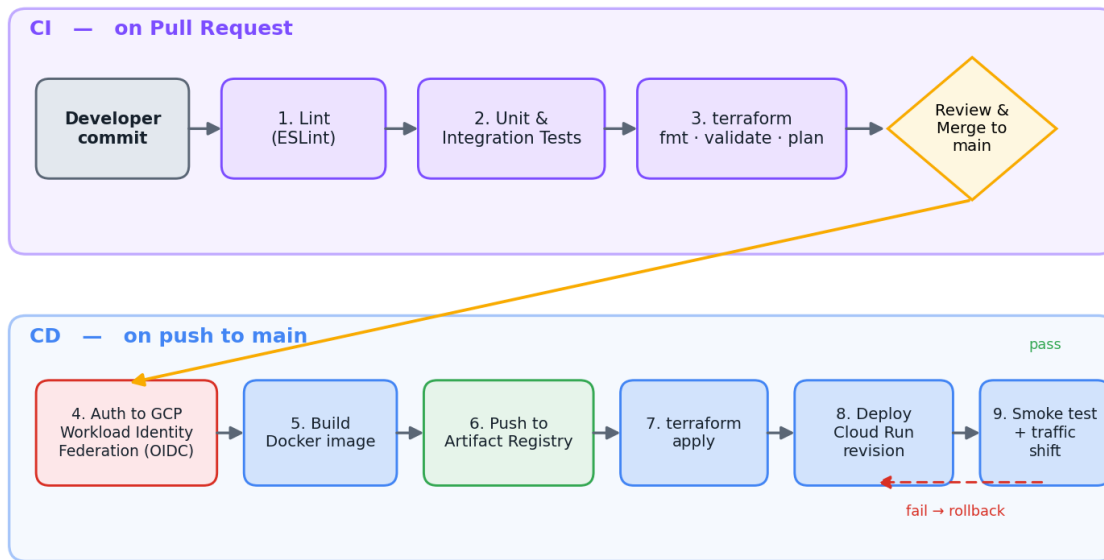


Figure 5.1: GitHub Actions CI/CD pipeline — validation on pull request, delivery on merge to main.

## 5.2 Pipeline Stages

| Stage        | Trigger      | Action                                           | Tooling                |
|--------------|--------------|--------------------------------------------------|------------------------|
| Lint         | Pull request | Static analysis of application code.             | ESLint                 |
| Test         | Pull request | Run unit and integration tests.                  | Jest / npm test        |
| Plan         | Pull request | Format, validate, and preview infra changes.     | terraform plan         |
| Authenticate | Push to main | Exchange GitHub OIDC token for GCP access.       | Workload Identity Fed. |
| Build        | Push to main | Build Docker image tagged with commit SHA.       | Docker buildx          |
| Push         | Push to main | Upload image to Artifact Registry.               | gcloud / docker push   |
| Apply        | Push to main | Converge infrastructure to desired state.        | terraform apply        |
| Deploy       | Push to main | Roll out a new Cloud Run revision.               | gcloud run deploy      |
| Verify       | Push to main | Smoke-test the new revision, then shift traffic. | curl / gcloud          |

## 5.3 Deployment Strategy

Cloud Run is revision-based: each deploy creates a new immutable revision while the previous one remains intact. The pipeline deploys the new revision with no traffic, runs a smoke test against its direct URL, and only then shifts 100% of traffic to it. If the smoke test fails, traffic stays on the last good revision and the deploy is effectively a no-op — an instant, built-in rollback. Because images are tagged

with the git commit SHA, every revision is traceable to an exact commit, and a rollback is simply re-pointing traffic at an earlier revision.

For the academic scope a single production environment is used. The same Terraform and workflows can stand up a separate staging project by changing one variable, should the team want a pre-production target later.

## 5.4 Keyless Authentication (Workload Identity Federation)

The pipeline never stores a service-account key. Instead, GitHub issues a short-lived OIDC token for each run; GCP's Workload Identity Federation trusts that token and lets the run impersonate a dedicated deployer service account for a few minutes. This removes the single most common CI/CD credential risk — a long-lived JSON key leaking from repository secrets. Only non-sensitive identifiers (project ID, the WIF provider resource name, and the deployer SA e-mail) are stored as GitHub Actions variables.

Illustrative authentication step in the CD workflow:

```
- name: Authenticate to Google Cloud (keyless)
  uses: google-github-actions/auth@v2
  with:
    project_id: ${vars.GCP_PROJECT_ID}
    workload_identity_provider: ${vars.WIF_PROVIDER}
    service_account: ${vars.DEPLOYER_SA}

- name: Deploy to Cloud Run
  run: |
    gcloud run deploy spms \
      --image=$REGION-docker.pkg.dev/$PROJECT/spms/app:$GITHUB_SHA \
      --region=$REGION --no-traffic --tag=candidate
```

## 6. Cost Analysis & Budget Management

### 6.1 Estimated Monthly Cost

The table below estimates steady-state monthly cost for low, demo-level traffic in the us-central1 (Tier-1) region. Figures are planning estimates, not quotes; actual spend depends on usage and should be confirmed with the GCP Pricing Calculator.

| Resource          | Configuration                      | Est. / month | Notes                                         |
|-------------------|------------------------------------|--------------|-----------------------------------------------|
| Cloud Run         | 1 vCPU/512 MiB, min=0, low traffic | \$0 – 3      | Mostly inside free tier (2M req, 180k vCPU-s) |
| Cloud SQL (MySQL) | db-f1-micro, 10 GB SSD             | \$9 – 10     | Largest line item; shared-core, no SLA        |
| Cloud Storage     | < 5 GB standard + state            | ~ \$0.20     | Negligible at this scale                      |
| Artifact Registry | < 2 GB of images                   | ~ \$0.10     | Near the free storage allotment               |
| Secret Manager    | ~6 secrets, low access             | < \$1        | \$0.06 / active version + access              |



| Resource             | Configuration             | Est. / month | Notes                           |
|----------------------|---------------------------|--------------|---------------------------------|
| Networking egress    | minimal; same-region free | \$0 – 1      | 1 GiB North-America egress free |
| Logging & Monitoring | within free allotment     | ~ \$0        | First 50 GiB of logs free       |
| Estimated total      | —                         | ≈ \$11 – 15  | ≈ \$22 – 30 over two months     |

**Headroom.** At roughly \$11–15 per month, two months of operation costs about \$22–30 — under 10% of the \$300 credit. The dominant cost is Cloud SQL, which can be stopped between work sessions to push spend lower still.

## 6.2 Cost Controls

- ☐ Scale to zero: Cloud Run min-instances=0 means no charge while the app is idle.
- ☐ No idle connector: Direct VPC egress avoids the always-on cost of a Serverless VPC connector.
- ☐ Stop Cloud SQL between sessions and start it only when developing or demoing.
- ☐ A Terraform-managed budget alerts the team at 50%, 90%, and 100% of the \$300 credit.
- ☐ Single Tier-1 region keeps inter-service data transfer free and compute rates at their lowest.
- ☐ Bucket lifecycle rules expire old objects; one terraform destroy returns all spend to zero after grading.

## 7. Motivation & Rationale

Each major decision favoured the option that delivered the most managed capability for the least operational and financial overhead, given an academic timeline and a fixed credit.

| Decision       | Chosen option         | Why (and what was set aside)                                                                                                                                                             |
|----------------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cloud platform | Google Cloud Platform | Free-trial credit is available to the team; strong managed services and first-class Terraform support. AWS/Azure offered no credit here.                                                 |
| Compute        | Cloud Run             | Scales to zero, no cluster to operate, cheapest for bursty low traffic, and it runs our container directly. GKE was overkill and costly; a plain VM means manual ops and always-on cost. |
| Database       | Cloud SQL for MySQL   | Managed backups, patching, and encryption on a cheap shared-core tier. Self-managing MySQL on a VM adds an ops burden with no benefit at this scale.                                     |
| Infra as Code  | Terraform             | Declarative, reproducible, and trivially destroyable; mature GCP provider and broad team familiarity. Hand-run gcloud scripts are not reproducible.                                      |
| CI/CD          | GitHub Actions        | Code already lives on GitHub; tight pull-request integration and ample free minutes for this scope, with native OIDC support. Cloud Build would split tooling across platforms.          |

| Decision      | Chosen option                | Why (and what was set aside)                                                                                       |
|---------------|------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Pipeline auth | Workload Identity Federation | Eliminates long-lived service-account keys — the highest-risk CI/CD secret — in favour of short-lived OIDC tokens. |

## 8. Points of Consideration

### 8.1 Secrets Hosting

All secrets live in Secret Manager and are referenced by the Cloud Run service at start-up under its own service account; they never appear in the repository, the Docker image, or environment files committed to git. The pipeline itself holds no secret key — it authenticates through Workload Identity Federation. Secrets can be rotated by adding a new secret version without redeploying code.

### 8.2 Security & Least Privilege

The database has no public IP and is reachable only through the VPC. The pipeline and the runtime use separate service accounts, each granted only the roles it needs. Public access to Cloud Run is restricted to the application's own endpoints over HTTPS. These boundaries reinforce SecureVault's zero-knowledge design, in which the platform stores only encrypted data.

### 8.3 Cost & Teardown

The budget alert and scale-to-zero defaults guard against runaway spend, and the whole environment is disposable: terraform destroy removes every resource in minutes, which is the safest way to protect the remaining credit once the project is graded.

### 8.4 Backups & Recovery

Cloud SQL is configured with automated daily backups and point-in-time recovery, so the database can be restored after an accidental change. The Terraform state bucket is versioned, allowing infrastructure state to be rolled back as well.

### 8.5 Region & Data Residency

us-central1 is chosen for its Tier-1 pricing and free-tier eligibility. If Canadian data residency becomes a requirement, the same Terraform can target northamerica-northeast1 (Montreal) by changing one variable, at a modestly higher rate.

### 8.6 Availability & Known Limits

The shared-core db-f1-micro instance is not covered by the Cloud SQL SLA, which is acceptable for an academic demo but would be upgraded for production. Cloud Run cold starts can add a brief delay on the first request after idle; setting min-instances to 1 during a live demonstration removes this at a small cost.

## 8.7 Risks & Mitigations

| Risk                                   | Mitigation                                                                     |
|----------------------------------------|--------------------------------------------------------------------------------|
| Free credit exhausted before term ends | Budget alerts at 50/90/100%; stop Cloud SQL when idle; teardown after grading. |
| Cold-start latency during a demo       | Temporarily set Cloud Run min-instances=1 for the demo window.                 |
| Secret or key leakage                  | Secret Manager + Workload Identity Federation; no keys in repo or image.       |
| Terraform state corruption             | Versioned, locked GCS remote state; restore a previous version.                |
| Configuration drift / bad change       | terraform plan gate on every pull request before merge.                        |
| Accidental data loss                   | Automated Cloud SQL backups with point-in-time recovery.                       |

## 9. Summary

This milestone defined a complete, cost-aware deployment design for SecureVault on Google Cloud Platform. The application is containerised and run on Cloud Run, backed by Cloud SQL for MySQL, Cloud Storage, and Secret Manager, with every resource declared in Terraform and shipped through a GitHub Actions pipeline that authenticates to GCP without stored keys.

The design keeps estimated spend near \$22–30 over the two-month window — well inside the \$300 credit — while preserving the security boundaries SecureVault requires: a private database, least-privilege identities, centralised secrets, and a fully reproducible, disposable environment. Together with the project plan and requirements analysis from earlier milestones, it provides a clear, defensible path from source commit to a running, secure system.

## References

- Google Cloud. (2026). Cloud Run pricing. <https://cloud.google.com/run/pricing>
- Google Cloud. (2026). Cloud SQL pricing. <https://cloud.google.com/sql/pricing>
- Google Cloud. (2026). About instance settings — Cloud SQL for MySQL. <https://cloud.google.com/sql/docs/mysql/instance-settings>
- Google Cloud. (2026). Best practices for cost-optimized Cloud Run services. <https://cloud.google.com/run/docs/tips/services-cost-optimization>
- Google Cloud. (2026). Secret Manager documentation. <https://cloud.google.com/secret-manager/docs>
- Google Cloud. (2026). Workload Identity Federation. <https://cloud.google.com/iam/docs/workload-identity-federation>
- HashiCorp. (2026). Terraform — Google Cloud provider documentation. <https://registry.terraform.io/providers/hashicorp/google/latest/docs>
- GitHub. (2026). GitHub Actions documentation. <https://docs.github.com/actions>
- google-github-actions. (2026). auth — Authenticate to Google Cloud from GitHub Actions. <https://github.com/google-github-actions/auth>

- National Institute of Standards and Technology. (2020). SP 800-63B: Digital identity guidelines. <https://pages.nist.gov/800-63-3/sp800-63b.html>
- OWASP Foundation. (2025). Password storage cheat sheet. <https://owasp.org>

## Part IV — System Test Plan

|                             |                                                                                                                                     |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Test Plan Identifier</b> | <b>TP-SecureVault-STP-V1.0</b>                                                                                                      |
| <b>Project</b>              | SecureVault – Secure Password Management System                                                                                     |
| <b>Course</b>               | PRG800 – Systems Analysis and Design                                                                                                |
| <b>Date</b>                 | June 28, 2026                                                                                                                       |
| <b>Version</b>              | 1.0 – Initial Release                                                                                                               |
| <b>Sponsor</b>              | Professor Miles McDonald                                                                                                            |
| <b>Group Members</b>        | Felix Akuetteh Torto   Akshay Nedumparambil Unnikrishnan<br>Harmon Anthony Tuazon   Jean Luc Sita Mbuya<br>Kulsum Timol   Anju Babu |

## 1. Test Plan Identifier

---

TP-SecureVault-STP-V1.0

## 2. References

---

- IEEE 829 – Standard for Software and System Test Documentation
- PRG800 Milestone 1 – Project Proposal and Group Contract (May 11, 2026)
- PRG800 Milestone 2 – Project Plan and Feasibility Analysis (May 26, 2026)
- PRG800 Milestone 3 – Requirements Analysis (June 9, 2026): Use Cases UC-01 to UC-05, Domain Class Diagram, NFRs
- NIST SP 800-63B – Digital Identity Guidelines: Authentication and Lifecycle Management
- OWASP Testing Guide v4.2 – Web Application Security Testing
- ISO/IEC 27001:2022 – Information Security Management Systems

## 3. Introduction

---

This is the system test plan for the SecureVault – Secure Password Management System, developed as a group academic project for PRG800 – Systems Analysis and Design at Seneca Polytechnic.

The purpose of this plan is to define the scope, objectives, approach, resources, and schedule for verifying the functional and security requirements of SecureVault. The system is a web-based, zero-knowledge password manager that allows individual users and students to securely store, generate, and manage passwords and sensitive documents within an AES-256 encrypted vault, protected by a hashed master password and Two-Factor Authentication (2FA).

Testing will verify all five core use cases identified in Milestone 3: Log In (UC-01), Add Credential (UC-02), Retrieve Credential (UC-03), Store Sensitive Document (UC-04), and Analyze Password Health & Notify (UC-05). In addition, non-functional security requirements — including zero-knowledge architecture, TLS in transit, and append-only audit logging — will be independently validated through dedicated security test cases.

The testing period is scheduled from July 1 to July 25, 2026. All defects will be managed and tracked in Bugzilla. Testing will be conducted using the Chrome desktop browser as the primary environment on the project's Node.js/Express/MySQL stack.

## 4. Test Items

---

The following modules and features of the SecureVault system are in scope for system testing:

- ☐ Authentication Module – User registration, login with master password, 2FA enrollment and verification, account lockout, and session expiry.
- ☐ Vault Management Module – Add, list, retrieve, and delete encrypted credentials; vault lock/unlock; user data isolation.
- ☐ Password Generator Module – Strong password generation with configurable length and character type rules.
- ☐ Password Health Module – Health report generation, weak password detection, password reuse detection, and security alert creation.
- ☐ Secure Documents Module – Upload, store (encrypted), retrieve, and validate file type/size restrictions for sensitive documents.
- ☐ Audit Log Module – Append-only action logging with timestamps and IP addresses; admin view.
- ☐ Encryption & Security – AES-256 encryption at rest, TLS in transit, hashed master password storage, and zero-knowledge server-side validation.
- ☐ User Interface – Navigation, responsive layout, and performance benchmarks.

## 5. Software Risk Issues

---

- ☐ **Zero-knowledge encryption dependency: The AES-256-GCM encryption module is critical. Any regression in this component would expose plaintext data, constituting a Critical defect.**
- ☐ **2FA integration is partially implemented: UC-01 and its 2FA sub-flow may be blocked until the TOTP sprint completes. Test cases TC-AUTH-09 and TC-AUTH-10 are contingent on this.**
- ☐ Secure Document module (UC-04) is marked 'To Build': TC-DOC-\* test cases will be deferred until the feature is implemented.
- ☐ Short academic testing window: The test cycle must complete within approximately four weeks, limiting time for deep exploratory testing and regression cycles.
- ☐ Mixed team experience levels: Some team members are new to security testing; targeted training and pair testing will be employed.

## 6. Features to be Tested

---

The table below summarizes the features under test and their associated risk levels. Risk Level H = High, M = Medium, L = Low.

| Feature                                  | Description                                                                                                              | Risk |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|------|
| User Registration and Login              | Users must be able to create an account with a strong master password and log in successfully using credentials and 2FA. | H    |
| Account Lockout after 5 Failed Attempts  | After 5 failed login attempts, the account must be locked for 15 minutes to prevent brute-force attacks.                 | H    |
| Vault Credential Add / Retrieve / Delete | Users must be able to add, view, and remove credentials; all data must be encrypted client-side before storage.          | H    |
| Password Generator                       | Users must be able to generate strong, configurable passwords meeting the defined character rules.                       | M    |
| Password Health Report and Alerts        | The system must scan stored passwords, flag weak and reused ones, and notify users via security alerts.                  | H    |
| Secure Document Upload and Retrieval     | Users must be able to upload PDFs and images (up to 10 MB) and retrieve them in decrypted form.                          | H    |
| Audit Log (Append-Only)                  | All key actions must be logged with timestamps; no user or admin may delete or edit audit entries.                       | H    |
| AES-256 Encryption at Rest               | All vault data and documents must be stored as ciphertext; no plaintext credentials visible in the database.             | H    |
| Zero-Knowledge Architecture              | The server must never hold or access plaintext master passwords or decrypted vault data.                                 | H    |
| Session Auto-Lock (10-minute idle)       | The vault must auto-lock and the session must expire after 10 minutes of inactivity.                                     | M    |
| User Data Isolation                      | A logged-in user must not be able to access any other user's credentials via direct API calls.                           | H    |
| UI Navigation and Performance            | All major sections must load and respond within 2 seconds; vault unlock must complete under 2 seconds.                   | M    |

## 7. Features Not to be Tested

- ☐ **Browser extension integration – Out of scope per Milestone 3 product scope definition.**



- ❑ **Enterprise IAM and third-party enterprise integrations – Explicitly excluded from scope.**
- ❑ **AI-based threat detection – Not part of the current system design.**
- ❑ **Commercial deployment and cloud enterprise infrastructure – Academic prototype only.**
- ❑ **Payment or banking functionality – Not applicable to this system.**
- ❑ Non-Chrome browsers (Firefox, Safari, Edge) – Testing is scoped to Chrome desktop in this cycle; cross-browser coverage recommended for future releases.
- ❑ Real-time enterprise monitoring dashboards – Not in scope.

## 8. Approach

---

### 8.1 Test Levels

Testing will be performed at the system and integration levels. The full, deployed SecureVault application — frontend (React/Vite), backend (Node.js/Express), and database (MySQL) — will be tested end-to-end as a complete system. Unit testing of individual functions (e.g., the AES-256 encryption utility) is the responsibility of developers and is out of scope for this test plan. No performance load testing is included.

### 8.2 Test Techniques

- ❑ Black-box functional test case execution: testers interact with the system through the web UI and REST API without access to source code.
- ❑ Boundary value analysis: applied to password length rules (minimum 12 characters), file size limits (10 MB), and lockout threshold (5 attempts).
- ❑ Negative testing: invalid inputs, unauthorized access attempts, and out-of-scope file types.
- ❑ Security testing: database inspection for plaintext credential exposure; API-level authorization bypass attempts; TLS traffic inspection.
- ❑ Exploratory testing: unscripted exploration of navigation flows and edge cases in the UI.

### 8.3 Test Tools

- ❑ Bugzilla – Defect tracking and reporting.
- ❑ Postman – API-level testing for backend endpoints (auth, vault, documents).
- ❑ Chrome DevTools – Network inspection, TLS verification, and response time measurement.
- ❑ MySQL Workbench – Database inspection to verify encryption at rest.
- ❑ GitHub – Version control for test artifacts and defect traceability.

## 8.4 Metrics Collected

| Metric                                                    | Collection Method / Notes                                         |
|-----------------------------------------------------------|-------------------------------------------------------------------|
| Test Cases Executed / Passed / Failed / Blocked           | Tracked daily; reported in daily execution report                 |
| Defect Count by Severity (Critical / High / Medium / Low) | Logged in Bugzilla; reviewed at each triage meeting               |
| Defect Turnaround Time                                    | Time from defect logged to developer fix to QA retest completion  |
| Regression Defect Count                                   | New defects introduced by fixes; tracked across regression cycles |
| Security Test Pass Rate (TC-SEC-*)                        | All security cases must pass before final sign-off                |
| Test Case Coverage (% of Use Cases Covered)               | Maps TCs to UC-01 through UC-05 from Milestone 3                  |

## 8.5 Configuration Management

Single test configuration:

- ☐ Hardware: Standard PC / laptop running Windows 10 or macOS
- ☐ Browser: Google Chrome (latest stable release) on desktop
- ☐ Backend: Node.js 20+ / Express running on localhost:4000
- ☐ Database: MySQL 8+ with schema from database/schema.sql
- ☐ Environment: Project test environment (configured per project README setup steps)

## 8.6 Regression Testing

Regression testing will be executed after each developer fix cycle. Failed test cases will be re-executed, with priority given to modules containing Critical or High severity defects. Security test cases (TC-SEC-\*) must be re-run in full after any change to the encryption, authentication, or audit log modules.

## 8.7 Special Requirements

- ☐ All TC-SEC-\* (Security) test cases require database read access and must be conducted by the Security Reviewer and Database Administrator jointly.

- ☐ TC-AUTH-09 and TC-AUTH-10 (2FA) are dependent on the 2FA implementation sprint being complete; they will be blocked and tracked in Bugzilla until then.
- ☐ TC-DOC-\* (Secure Documents) are blocked until UC-04 implementation is complete.
- ☐ All testing must use the HTTPS endpoint; HTTP-only access should not be used for test execution.

## 8.8 Organisational Processes

- ☐ Daily defect triage meetings between QA Lead, testers, and developers.
- ☐ Weekly progress review with the Project Manager to assess metrics, risks, and milestone readiness.
- ☐ GitHub used for all test artifact version control; Bugzilla for all defect records.
- ☐ QA Lead communicates test status and blockers to Prof. McDonald at each milestone checkpoint.

## 9. Item Pass/Fail Criteria

| Verdict        | Criteria                                                                                      |
|----------------|-----------------------------------------------------------------------------------------------|
| <b>PASS</b>    | Expected result matches actual result with no unresolved Critical or High defects.            |
| <b>FAIL</b>    | Actual result differs from expected result, or a Critical / High defect remains open.         |
| <b>BLOCKED</b> | Test case cannot be executed due to a prerequisite failure or missing feature implementation. |

All Critical and High severity defects must be resolved and verified by regression before the final test summary report is submitted. The project will be considered ready for final submission when 100% of executable High-priority test cases have passed.

## 10. Suspension Criteria and Resumption Requirements

Testing will be suspended if any of the following conditions occur:

- ☐ More than 70% of all test cases fail in a single execution cycle.
- ☐ A Critical defect is found in the AES-256 encryption module that exposes plaintext data.
- ☐ The test environment becomes unavailable for more than 24 consecutive hours.
- ☐ The authentication module fails completely, blocking all other test cases.

Testing will resume when:

- ☐ Developers confirm the blocking defect(s) are resolved and a new build is deployed.
- ☐ The Environment Manager confirms the test environment is stable.
- ☐ The QA Lead approves resumption after a smoke test pass on the core authentication and vault add/retrieve flows.

## 11. Test Deliverables

---

- ☐ System Test Plan document (this document)
- ☐ Test Case Specifications — all TC-\* cases with test data and expected results
- ☐ Daily Execution Reports — test case status summaries from each test session
- ☐ Defect Reports — all issues logged, triaged, and tracked in Bugzilla
- ☐ Regression Test Results — outcomes of re-testing after defect fixes
- ☐ Final Test Summary Report — overall findings, metrics, open defects, and test coverage assessment

## 12. Remaining Test Tasks

---

The following tasks remain to be completed after this test plan is approved:

- ☐ A. Create and peer-review detailed test cases for all modules (TC-AUTH-\*, TC-VAULT-\*, TC-PWGEN-\*, TC-HEALTH-\*, TC-DOC-\*, TC-AUDIT-\*, TC-SEC-\*, TC-UI-\*).
- ☐ B. Set up and validate the test environment (Node.js, MySQL, HTTPS, Chrome) per the project README.
- ☐ C. Execute all test cases according to the schedule defined in Section 16.
- ☐ D. Log all defects in Bugzilla and conduct daily triage meetings throughout the cycle.
- ☐ E. Conduct regression testing after each developer fix cycle.
- ☐ F. Prepare and submit the Final Test Summary Report by July 25, 2026.

## 13. Test Cases

---

The table below contains all 48 planned test cases covering all in-scope modules. TC IDs follow the format TC-[MODULE]-[NUMBER]. Priority: H = High, M = Medium, L = Low.

| TC ID      | Module         | Feature           | Test Description                  | Expected Result                                | Priority | Test Type  |
|------------|----------------|-------------------|-----------------------------------|------------------------------------------------|----------|------------|
| TC-AUTH-01 | Authentication | User Registration | Register with valid email, strong | Account created, verification email sent, user | High     | Functional |

| TC ID      | Module         | Feature           | Test Description                                                        | Expected Result                                                            | Priority | Test Type  |
|------------|----------------|-------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------|----------|------------|
|            |                |                   | master password (≥12 chars, mixed types), and matching confirm password | redirected to login                                                        |          |            |
| TC-AUTH-02 | Authentication | User Registration | Attempt registration with master password shorter than 12 characters    | Error message displayed: 'Password must be at least 12 characters'         | High     | Negative   |
| TC-AUTH-03 | Authentication | User Registration | Attempt registration with master password lacking mixed character types | Error displayed requiring uppercase, lowercase, number, and symbol         | High     | Negative   |
| TC-AUTH-04 | Authentication | User Registration | Attempt registration with an already-registered email address           | Error: 'An account with this email already exists'                         | Medium   | Negative   |
| TC-AUTH-05 | Authentication | Login             | Login with valid credentials and correct 2FA code (TOTP)                | JWT session token issued; vault unlocked; login event written to audit log | High     | Functional |
| TC-AUTH-06 | Authentication | Login             | Login with correct email but wrong master password                      | Access denied; failed-attempt counter incremented in DB                    | High     | Negative   |

| TC ID      | Module         | Feature          | Test Description                                                     | Expected Result                                                     | Priority | Test Type  |
|------------|----------------|------------------|----------------------------------------------------------------------|---------------------------------------------------------------------|----------|------------|
| TC-AUTH-07 | Authentication | Account Lockout  | Submit 5 consecutive incorrect master passwords for the same account | Account locked for 15 minutes; lockout message shown to user        | High     | Functional |
| TC-AUTH-08 | Authentication | Account Lockout  | Attempt login on a locked account before lockout period expires      | Login rejected with 'Account temporarily locked' message            | High     | Functional |
| TC-AUTH-09 | Authentication | 2FA Enrollment   | Enroll TOTP 2FA by scanning QR code and verifying first code         | 2FA enabled; TwoFactorConfig record saved; user notified of success | High     | Functional |
| TC-AUTH-10 | Authentication | 2FA Verification | Login from a new device without submitting 2FA code                  | Login blocked; 2FA prompt shown; session not created                | High     | Security   |
| TC-AUTH-11 | Authentication | Session Expiry   | Leave vault idle for more than 10 minutes without any interaction    | Session expired; vault auto-locked; user returned to login screen   | High     | Functional |
| TC-AUTH-12 | Authentication | Session Expiry   | Attempt to access vault API endpoint with an expired JWT token       | 401 Unauthorized returned; request rejected                         | High     | Security   |
| TC-VAULT-  | Vault          | Add              | Add a new credential                                                 | Credential encrypted (AES-                                          | High     | Functional |

| TC ID       | Module           | Feature             | Test Description                                                           | Expected Result                                                                 | Priority | Test Type  |
|-------------|------------------|---------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------|----------|------------|
| 01          | Management       | Credential          | with site URL, username, and password while logged in                      | 256-GCM) and stored; add action written to audit log                            |          |            |
| TC-VAULT-02 | Vault Management | Add Credential      | Attempt to save a new credential with the required 'site' field left empty | Save blocked; validation error: 'Site URL is required'                          | High     | Negative   |
| TC-VAULT-03 | Vault Management | Retrieve Credential | Select an existing credential entry from the vault list                    | Credential decrypted on the client; plaintext displayed; retrieve action logged | High     | Functional |
| TC-VAULT-04 | Vault Management | Retrieve Credential | Attempt to retrieve a credential that has been deleted from the vault      | Error message: 'Credential not found'; no data returned                         | Medium   | Negative   |
| TC-VAULT-05 | Vault Management | Delete Credential   | Delete an existing credential entry from the vault                         | Credential removed from DB; delete action written to audit log                  | High     | Functional |
| TC-VAULT-06 | Vault Management | List Credentials    | View the vault after adding multiple credentials                           | All stored credentials listed; no plaintext passwords visible in list view      | High     | Functional |

| TC ID       | Module             | Feature           | Test Description                                                                                    | Expected Result                                                                     | Priority | Test Type  |
|-------------|--------------------|-------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|----------|------------|
| TC-VAULT-07 | Vault Management   | Data Isolation    | User A attempts to access or retrieve User B's credential via API (e.g., by guessing credential ID) | 403 Forbidden returned; User B's data not accessible to User A                      | High     | Security   |
| TC-VAULT-08 | Vault Management   | Vault Lock        | Manually lock the vault using the lock button                                                       | Session cleared; vault locked; user redirected to login screen                      | Medium   | Functional |
| TC-PWGEN-01 | Password Generator | Generate Password | Request a password of length 16 with uppercase, lowercase, numbers, and symbols all enabled         | Generated password of exactly 16 characters containing all required character types | High     | Functional |
| TC-PWGEN-02 | Password Generator | Generate Password | Request a password of length 8 (minimum) with only lowercase letters selected                       | 8-character password containing only lowercase letters returned                     | Medium   | Functional |
| TC-PWGEN-03 | Password Generator | Generate Password | Generate two passwords with identical settings back-to-back                                         | Two different (non-identical) passwords returned, demonstrating randomness          | Medium   | Functional |
| TC-PWGEN-04 | Password Generator | Copy to Clipboard | Click 'Copy' on a generated                                                                         | Password copied to clipboard;                                                       | Low      | Functional |



| TC ID               | Module          | Feature                       | Test Description                                                                                  | Expected Result                                                                      | Priority | Test Type  |
|---------------------|-----------------|-------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|----------|------------|
|                     |                 |                               | password                                                                                          | confirmation message displayed to user                                               |          |            |
| <b>TC-HEALTH-01</b> | Password Health | Health Report                 | Open the health report with at least one credential stored in the vault                           | Report generated; each credential assigned a strength score; overall score displayed | High     | Functional |
| <b>TC-HEALTH-02</b> | Password Health | Weak Password Alert           | Store a credential with a short, simple password (e.g., '123456') and run the health report       | Credential flagged as 'Weak'; SecurityAlert created; user notified in dashboard      | High     | Functional |
| <b>TC-HEALTH-03</b> | Password Health | Reuse Alert                   | Store two credentials with identical passwords and run the health report                          | Both credentials flagged as 'Reused'; alerts raised for each reuse instance          | High     | Functional |
| <b>TC-HEALTH-04</b> | Password Health | Reuse Detection – 30-Day Rule | Store a credential, delete it, re-add the same password within 30 days, and run the health report | New entry flagged as reused within 30-day window per business rule                   | Medium   | Functional |
| <b>TC-HEALTH-05</b> | Password Health | Empty Vault Report            | Open the health report when the vault contains no credentials                                     | Report shows 'No credentials found'; no errors thrown                                | Low      | Functional |

| TC ID       | Module           | Feature               | Test Description                                                           | Expected Result                                                                  | Priority | Test Type  |
|-------------|------------------|-----------------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------|----------|------------|
| TC-DOC-01   | Secure Documents | Upload Document       | Upload a valid PDF file under 10 MB while logged in                        | File encrypted (AES-256) and stored; upload action written to audit log          | High     | Functional |
| TC-DOC-02   | Secure Documents | Upload Document       | Upload a valid image file (PNG/JPG) under 10 MB                            | Image encrypted and stored; upload event logged in audit log                     | High     | Functional |
| TC-DOC-03   | Secure Documents | File Type Restriction | Attempt to upload a file with an unsupported extension (e.g., .exe, .zip)  | Upload blocked; error: 'Only PDF and image files are supported'                  | High     | Negative   |
| TC-DOC-04   | Secure Documents | File Size Restriction | Attempt to upload a PDF exceeding 10 MB                                    | Upload blocked; error: 'File size must not exceed 10 MB'                         | High     | Negative   |
| TC-DOC-05   | Secure Documents | Retrieve Document     | Open a previously uploaded document from the vault                         | Document decrypted on client side; file downloads or renders correctly           | High     | Functional |
| TC-AUDIT-01 | Audit Log        | Log Generation        | Perform login, add credential, retrieve credential, and logout in sequence | Audit log shows all four events with correct timestamps, user ID, and IP address | High     | Functional |
| TC-AUDIT-02 | Audit Log        | Append-Only Integrity | Attempt to delete or modify an existing audit log                          | Operation rejected; 403 or 405 returned; log entry unchanged                     | High     | Security   |

| TC ID              | Module     | Feature                     | Test Description                                                                            | Expected Result                                                                            | Priority | Test Type  |
|--------------------|------------|-----------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|----------|------------|
|                    |            |                             | entry via the API                                                                           |                                                                                            |          |            |
| <b>TC-AUDIT-03</b> | Audit Log  | Admin Access                | System administrator views the audit log for a specific user account                        | Full audit history displayed with all logged actions and metadata                          | Medium   | Functional |
| <b>TC-SEC-01</b>   | Encryption | AES-256 at Rest             | Inspect the database credentials table after adding a credential                            | Stored password field contains ciphertext only; no plaintext password visible in DB        | High     | Security   |
| <b>TC-SEC-02</b>   | Encryption | Master Password Hashing     | Inspect the users table after account registration                                          | Only the hashed value of the master password is stored; no plaintext master password in DB | High     | Security   |
| <b>TC-SEC-03</b>   | Encryption | TLS in Transit              | Intercept network traffic between client and server during a credential retrieval           | All data in transit is encrypted via TLS; no plaintext credential visible in packets       | High     | Security   |
| <b>TC-SEC-04</b>   | Encryption | Zero-Knowledge Architecture | Review server-side log files and DB records for any master password or decrypted vault data | No plaintext master passwords or decrypted vault contents found server-side                | High     | Security   |

| TC ID    | Module         | Feature           | Test Description                                                                     | Expected Result                                                                      | Priority | Test Type   |
|----------|----------------|-------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|----------|-------------|
| TC-UI-01 | User Interface | Navigation        | Navigate to all main sections: Vault, Generator, Health Report, Documents, Audit Log | All sections load without errors; navigation responds in under 2 seconds             | Medium   | Functional  |
| TC-UI-02 | User Interface | Responsive Layout | Access the application on a mobile-width viewport (375 px)                           | Layout adjusts correctly; all core functions accessible without horizontal scrolling | Low      | Usability   |
| TC-UI-03 | User Interface | Performance       | Unlock vault containing 50+ credential entries                                       | Vault loads and becomes interactive in under 2 seconds                               | Medium   | Performance |

## 14. Environmental Needs

- ☐ Test Environment: Project-hosted Node.js/Express + MySQL stack per README setup instructions.
- ☐ Hardware: Standard Windows 10 or macOS PC/laptop with Chrome browser installed.
- ☐ Software: Node.js 20+, MySQL 8+, Postman, MySQL Workbench, Chrome DevTools.
- ☐ Network: HTTPS endpoint required for all security test cases (TC-SEC-\*).
- ☐ Data: A clean test database pre-seeded with at least two user accounts and sample credentials for isolation tests.
- ☐ Stable environment confirmed by Environment Manager before each test session.

## 15. Staffing and Training Needs

The testing team will include QA testers drawn from the SecureVault development group, supervised by a designated QA Lead.

Before test execution begins, all team members will receive a brief orientation covering:

- ❑ How each SecureVault module works (auth flow, vault encryption, health analysis, document upload).
- ❑ How to use Postman for API-level testing of backend endpoints.
- ❑ How to inspect the MySQL database to verify encryption at rest.
- ❑ How to log and triage defects in Bugzilla following the team's severity classification.
- ❑ Security testing fundamentals — what to look for when verifying zero-knowledge and AES-256 compliance.

No external or specialized automation training is required; all testing will be conducted manually with Postman and Chrome DevTools.

## 16. Responsibilities

| Role                   | Responsibilities                                                                                                                                    |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| QA Lead                | Prepares and reviews the test plan; assigns test cases; reviews daily reports; escalates defects; communicates with developers and project manager. |
| QA Testers             | Execute test cases per the approved plan; log all defects in Bugzilla; retest resolved defects; participate in daily triage.                        |
| Developers             | Fix reported defects within agreed SLA; support QA during triage; provide builds and deployment confirmation.                                       |
| Security Reviewer      | Validate security-specific test cases (TC-SEC-*); review encryption, hashing, and zero-knowledge compliance findings.                               |
| Database Administrator | Verify DB-level encryption at rest; confirm no plaintext credentials are stored; maintain test DB environment.                                      |
| Environment Manager    | Keep the test environment stable; configure Node.js, MySQL, and HTTPS; coordinate environment resets between test runs.                             |
| Project Manager        | Approves test plan; monitors test progress; authorises go/no-go decisions; coordinates with academic sponsor (Prof. McDonald).                      |

## 17. Schedule

All testing activities are scheduled within the Agile Sprint 5 (Testing & QA) and Sprint 6 (Finalization) windows defined in Milestone 2. Specific activities and owners are listed below.

| # | Task                                                                           | Planned Dates      | Owner                          |
|---|--------------------------------------------------------------------------------|--------------------|--------------------------------|
| A | Environment setup and validation                                               | Week 1 (Jul 1–5)   | Environment Manager            |
| B | Test plan review and sign-off                                                  | Week 1 (Jul 1–5)   | QA Lead / Project Manager      |
| C | Test case creation and peer review for Auth, Vault, Generator modules          | Week 1 (Jul 1–5)   | QA Team                        |
| D | Test case creation for Health, Documents, Audit, Security, UI modules          | Week 2 (Jul 7–11)  | QA Team                        |
| E | Execution – Authentication & Session (TC-AUTH-*)                               | Week 2 (Jul 7–11)  | QA Testers                     |
| F | Execution – Vault Management & Password Generator (TC-VAULT-*, TC-PWGEN-*)     | Week 2 (Jul 7–11)  | QA Testers                     |
| G | Execution – Password Health & Secure Documents (TC-HEALTH-*, TC-DOC-*)         | Week 3 (Jul 14–18) | QA Testers                     |
| H | Execution – Audit Log, Encryption/Security, UI (TC-AUDIT-*, TC-SEC-*, TC-UI-*) | Week 3 (Jul 14–18) | QA Testers / Security Reviewer |
| I | Defect triage, logging in Bugzilla, developer fix cycle                        | Ongoing Weeks 2–4  | QA Lead / Developers           |
| J | Regression testing after defect fixes                                          | Week 4 (Jul 21–25) | QA Team                        |
| K | Final Test Summary Report preparation and submission                           | Week 4 (Jul 25)    | QA Lead                        |

## 18. Planning Risks and Contingencies

| Risk                                        | Level  | Mitigation                                                                                               |
|---------------------------------------------|--------|----------------------------------------------------------------------------------------------------------|
| Limited test time (academic deadline)       | Medium | Prioritise High-risk TCs first; defer Low-priority cases if needed.                                      |
| Unstable Node.js / MySQL environment        | High   | Environment Manager confirms environment health before each test session; snapshot/restore strategy.     |
| Encryption library regression (AES-256-GCM) | High   | Run TC-SEC-* after every code push; automated smoke test on encryption module.                           |
| 2FA integration not yet fully implemented   | High   | Block TC-AUTH-09 / TC-AUTH-10 until 2FA sprint completes; use stub/mock for earlier cycles.              |
| Team member absence                         | Medium | QA Lead redistributes test cases; core TCs covered by at least two testers.                              |
| Delayed defect fixes from developers        | Medium | Triage daily; continue testing unblocked areas; escalate blockers to Project Manager.                    |
| Secure document module not yet built        | High   | TC-DOC-* deferred until UC-04 implementation; track via Bugzilla feature flag.                           |
| Browser / platform differences              | Low    | Testing scoped to Chrome desktop (primary) and one mobile breakpoint; note exclusions in summary report. |

## 19. Approvals

| Role                           | Name                     | Signature / Date |
|--------------------------------|--------------------------|------------------|
| Project Sponsor (Academic)     | Professor Miles McDonald |                  |
| Project Leader                 | Felix Torto              | Felix            |
| Research Analyst/Documentation | Akshay Nedumparambil     | Akshay           |
| PM/Developer/Programmer        | Harmon Tuazon            | Harmon           |

| Role                             | Name           | Signature / Date |
|----------------------------------|----------------|------------------|
| QA Tester/Developer              | Anju Babu      | Anju             |
| Programmer/Developer             | Kulsum Timol   | Kulsum           |
| Technical Lead/Database Designer | Jean Luc Mbuya | Jean             |

## 20. Glossary

| Term                               | Definition                                                                                                       |
|------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>AES-256-GCM</b>                 | Advanced Encryption Standard with 256-bit key in Galois/Counter Mode; used to encrypt all vault data at rest.    |
| <b>Audit Log</b>                   | An append-only record of all user and system actions, used for accountability and forensic review.               |
| <b>Bugzilla</b>                    | Open-source defect tracking tool used to log, manage, and report software issues throughout the test cycle.      |
| <b>2FA / TOTP</b>                  | Two-Factor Authentication using a Time-based One-Time Password algorithm (e.g., Google Authenticator, RFC 6238). |
| <b>E2EE</b>                        | End-to-End Encryption: data is encrypted on the client device before being transmitted to the server.            |
| <b>JWT</b>                         | JSON Web Token: a signed token issued at login, used to authenticate subsequent API requests.                    |
| <b>Master Password</b>             | The single passphrase the user creates at registration; never stored in plaintext — only its hash is retained.   |
| <b>TLS</b>                         | Transport Layer Security: cryptographic protocol that encrypts data in transit between client and server.        |
| <b>UC</b>                          | Use Case: a functional requirement narrative defined in Milestone 3 (UC-01 through UC-05).                       |
| <b>Zero-Knowledge Architecture</b> | A design principle where the server never holds, sees, or can reconstruct any plaintext user credentials.        |



| Term                      | Definition                                                                                                                    |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Vault</b>              | The user's encrypted container within SecureVault that stores all credentials and secure documents.                           |
| <b>Regression Testing</b> | Re-testing previously passing test cases after defect fixes to confirm no new issues were introduced.                         |
| <b>Smoke Testing</b>      | A brief preliminary test run to verify that basic system functions work before full test execution begins.                    |
| <b>Black-Box Testing</b>  | A testing approach where the tester interacts only with the system's external interfaces, without knowledge of internal code. |

## Summary

This document has presented the complete system design for SecureVault, carrying the project from the requirements established in Milestone 3 into a build-ready specification. The software design promoted the domain model into a layered architecture with dedicated controllers, data-access objects, and a single cryptographic service that enforces the zero-knowledge boundary, and it realised each of the five use cases as a sequence diagram. The user-interface design translated those same use cases into a coherent navigation structure, a security-critical login-and-2FA flow, and six annotated wireframes governed by established usability heuristics and a secure-by-default principle.

On the structural side, the database architecture mapped the domain model to a normalised (3NF) MySQL schema in which every secret is stored only as a salted hash or AES-256 ciphertext, and the deployment architecture specified a cost-aware, fully reproducible Google Cloud environment in which the database is never publicly exposed, secrets live only in Secret Manager, and the pipeline authenticates without any long-lived keys. The system test plan then defined how all of this will be verified, with 48 test cases spanning every use case and a dedicated set of security tests for encryption at rest, TLS in transit, append-only auditing, and the zero-knowledge guarantee.

Because every screen, class, table, and infrastructure boundary traces back to a documented requirement, the design is internally consistent and directly implementable. SecureVault is ready to proceed from design into development with a clear, secure, and testable foundation.